

Федеральное государственное образовательное бюджетное
учреждение высшего образования
**«Финансовый университет при Правительстве Российской Федерации»
(Финансовый университет)**

Факультет информационных технологий и анализа больших данных
Кафедра искусственного интеллекта

Выпускная квалификационная работа

на тему: «Разработка системы оценки сходства изображений для
визуального поиска»

Направление подготовки: 01.03.02 Прикладная математика и информатика

Профиль: Прикладное машинное обучение

Выполнил студент учебной группы

ПМ22-1

Трофимов Михаил Алексеевич

Руководитель доцент, к.ф.-м.н., доцент

Калажоков Заур Хамидбиевич

**ВКР соответствует предъявляемым
требованиям**

Заведующий кафедрой искусственного
интеллекта, к.э.н., доцент

_____М.В. Коротеев

«___» _____2026 г.

Москва 2026

Содержание

ВВЕДЕНИЕ	3
ГЛАВА 1. ИССЛЕДОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1. Разбор существующих решений и систем визуального поиска схожих изображений	6
1.2. Обзор метрик ранжирования и методов сравнения объектов в векторном пространстве	14
ГЛАВА 2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ ОЦЕНКИ СХОДСТВА ИЗОБРАЖЕНИЙ	21
2.1. Анализ архитектур нейронных сетей для генерации векторных представлений	21
2.2. Требования к системе поиска похожих изображений	47
ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ	53
3.1. Архитектура приложения для визуального поиска изображений	53
3.2. Экспериментальное исследование качества поиска и оценка работы системы	63
ЗАКЛЮЧЕНИЕ	73
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	75
ПРИЛОЖЕНИЕ 1. Примеры работы приложения	80
ПРИЛОЖЕНИЕ 2. Основной эксперимент	87

ВВЕДЕНИЕ

Разработка системы оценки сходства изображений для визуального поиска — задача поиска изображения по визуальному содержанию, или же Content-Based Image Retrieval (CBIR). Она заключается в том, чтобы на основе переданного изображения найти похожие. Термин был введен в употребление ещё в 1992 году японским исследователем Тошиказу Като для описания автоматического поиска изображений по цвету и форме [29]. Первой успешной реализацией такой системы стала коммерческая система Query by Image Content (QBIC) от IBM, описанная в 1995 году [26]. Для поиска похожих изображений использовались низкоуровневые визуальные характеристики: цветовые гистограммы, текстурные особенности и формы объектов. Пользователь мог задать запрос в виде другого изображения или схематического рисунка, после чего система находила изображения с похожими визуальными характеристиками. Однако низкоуровневые признаки содержали ряд недостатков: они обладали низкой устойчивостью к изменениям изображения, плохо масштабировались на сложные сцены и в целом имели ограниченную выразительность.

Значительный прогресс в решении задачи поиска изображений произошел с развитием методов глубокого обучения, прежде всего сверточных нейронных сетей и трансформерных архитектур. Эти методы позволили извлекать из изображений высокоуровневые представления — эмбединги, которые кодируют семантическую информацию о содержимом изображения. Современные крупные IT-компании, такие как Яндекс, Google и Microsoft, активно используют данные подходы в своих сервисах визуального поиска. Их сервисы: Яндекс Картинки, Google Images и Bing Images, позволяют находить не только идентичные изображения, но и визуально похожие объекты, их фрагменты и различные вариации одного и того же изображения. Они анализируют изображения в реальном времени и сопоставляют их с огромными базами данных.

Помимо глобальных поисковых платформ активно развиваются локальные системы управления фотоархивами, ориентированные на хранение и обработку пользовательских коллекций изображений. К таким решениям относятся, например, Immich, PhotoPrism и DigiKam. Данные системы позволяют пользователям хранить изображения на собственных серверах или персональных компьютерах, обеспечивая контроль над данными и приватностью. Они также обладают значительным функционалом, в том числе способны искать похожие изображения. Тем не менее современные локальные решения ориентированы преимущественно на пользовательское взаимодействие и не предназначены для исследовательских задач. В таких системах используются заранее встроенные модели извлечения признаков, а оценка качества поисковой выдачи носит субъективный характер. Отсутствие возможности подключать пользовательские модели и автоматически вычислять стандартные метрики информационного поиска, такие как precision@k , recall@k или Mean Average Precision (mAP), делает невозможным проведение экспериментальных исследований и сравнительный анализ различных подходов к поиску изображений.

Актуальность работы обусловлена необходимостью создания инструментов визуального поиска изображений, применимых не только в пользовательских сценариях, но и в исследовательской деятельности.

Цель работы — разработка локальной системы оценки сходства изображений для визуального поиска, обеспечивающей возможность загрузки собственных моделей и структурированных датасетов, например в формате `torchvision.datasets.ImageFolder`, для удобного подсчета метрик. Такое решение позволит не только оценивать эффективность различных моделей извлечения признаков, но и оперативно выявлять ошибки в разметке датасетов классификации изображений.

Для достижения цели были поставлены следующие задачи:

1. Проанализировать архитектурные подходы и функциональные возможности современных глобальных и локальных систем визуального поиска изображений;
2. Изучить методы сравнения объектов в векторном пространстве, основные метрики ранжирования, архитектуры нейронных сетей для генерации векторных представлений;
3. Разработать приложение для визуального поиска изображений, поддерживающее использование пользовательских моделей извлечения признаков и автоматический расчёт метрик качества ранжирования;
4. Обучить сверточную нейронную сеть для извлечения признаков изображений с нуля и подготовить её для использования в системе визуального поиска;
5. Провести серию экспериментов с разработанной системой, включая сравнение обученной с нуля модели с предобученными моделями извлечения признаков на основе метрик качества ранжирования.

Объектом исследования является процесс визуального поиска изображений в локальных коллекциях на основе их содержания. Предметом исследования является разработка и оценка системы визуального поиска изображений с использованием пользовательских моделей извлечения признаков и автоматического расчёта метрик качества ранжирования.

ГЛАВА 1. ИССЛЕДОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

В данной главе представлен обзор глобальных и локальных решений задачи CBIR. Глобальные решения представлены сервисами Яндекса, Google и Microsoft. Выбор Google Images и Bing Images обусловлен тем, что это сервисы крупнейших мировых поисковых систем, чьи браузеры и поисковые платформы охватывают подавляющую долю глобального интернет-трафика, что делает их подходы к визуальному поиску наиболее показательными для анализа современных промышленных решений. Яндекс Картинки представлены как безальтернативное отечественное глобальное решение.

Среди множества локальных решений были выбраны Immich, PhotoPrism и DigiKam. DigiKam — одно из старейших приложений для управления фотоархивами, появившееся ещё в 2001 году и использующее низкоуровневые визуальные характеристики для решения задачи CBIR. Immich и PhotoPrism являются современными self-hosted решениями, применяющими нейросетевой и гибридный подходы.

Во второй подглаве рассматриваются основные метрики информационного поиска и методы сравнения объектов в векторном пространстве. Архитектуры нейронных сетей, используемых для генерации векторных представлений изображений, в данной главе рассматриваются лишь обзорно, поскольку их подробный анализ представлен в отдельной главе работы.

1.1. Разбор существующих решений и систем визуального поиска схожих изображений

Современный подход к поиску похожих изображений в сервисе Яндекс Картинки в контексте CBIR можно описать как задачу поиска ближайших соседей в пространстве нейросетевых векторных представлений. В такой постановке каждое изображение представляется

компактным эмбедингом, а поиск похожих сводится к поиску наиболее близких векторов в большой базе. Подобный переход от классических локальных признаков к нейросетевым представлениям у Яндекса публично фиксируется как этап существенного улучшения поиска по загруженной картинке за счёт применения сверточных нейросетей, что повысило способность находить не только идентичные копии, но и семантически близкие изображения [21].

Использование эмбедингов для визуального поиска подтверждено и снимом Yandex visual search engine — это набор данных, в котором представлен 1 миллиард эмбедингов изображений размерности 200, полученных моделью Se-ResNext-101 [1]. Уже в конце 2021 года в поиске по картинкам находилось более 60 миллиардов изображений, что задавало требования к масштабируемости хранения и поиска [40]. В связи с этим существует основная инженерная проблема: поиск ближайших соседей в базе размером десятки миллиардов нельзя выполнять полным перебором. Поэтому в промышленной системе неизбежно применяется приближённый поиск ближайших соседей (Approximate Nearest Neighbor Search, ANN/ANNS) с использованием специализированных индексов. Яндекс не раскрывает, какими именно алгоритмами ANN пользуется, однако по опубликованным материалам Yandex Research видно, что компания активно работает в области ANNS и рассматривает различные решения, в том числе графовые алгоритмы, для обеспечения высокой скорости поиска без значительной потери точности [12].

Итого общая логика пайплайна заключается в том, что офлайн формируется база эмбедингов по индексируемым изображениям, а онлайн изображение-запрос преобразуется в эмбединг и далее выполняется быстрый поиск ближайших соседей по построенному индексу. Такой подход позволяет поддерживать поиск на больших коллекциях, сохраняя

приемлемую задержку ответа и масштабируя систему через распределение индекса.

Отдельный интерес представляет наличие в Яндекс Картинках специализированных функций, дополняющих задачу CBIR. Например, Яндекс развивает режим «Товары на картинке», в котором система выделяет объекты на изображении и сопоставляет их с товарами, чтобы найти такие же или очень похожие в магазинах [40]. По сути этот сценарий является расширением CBIR на уровень объектов внутри изображения: в качестве запроса может выступать не только всё изображение целиком, но и отдельные визуальные сущности, распознанные на нём.

Наконец, уточнённый пайплайн работы поиска по картинке можно описать через публичную документацию API [20]. Согласно документации, запрос может передаваться либо ссылкой на изображение (url), либо самим файлом в виде данных (data), либо через идентификатор запроса (id, CBIR ID), который возвращается в ответе и может использоваться повторно. Это указывает на существование механизма повторного обращения к результатам поиска и повторного использования внутреннего представления запроса. В документации также присутствует настройка familyMode, позволяющая управлять фильтрацией контента в выдаче, например, исключая нежелательные категории, что означает наличие отдельного этапа постобработки результатов поиска и применения политик контента поверх основной части. Таким образом, даже в рамках нейросетевого CBIR-подхода рассматриваемого глобального решения выдача формируется не только по близости эмбедингов, но и с учётом прикладных требований, включая фильтрацию контента и специализированные режимы поиска.

Современный подход к поиску схожих изображений в сервисе Google Images, как и в решении Яндекс Картинок, можно описать через задачу

поиска ближайших соседей в пространстве нейросетевых векторных представлений. В обеих системах изображение-запрос сначала преобразуется в эмбединг, а затем по нему выполняется быстрый поиск по заранее проиндексированной базе. Со стороны Google опубликовано открытое ANN-решение — собственная библиотека для эффективного поиска по векторному сходству ScaNN, ориентированная на работу с большими объёмами данных [15].

В открытых исследовательских работах компании также прослеживается развитие двухэтапной системы отбора кандидатов для задачи CBIR. В 2017 году была представлена статья Large-Scale Image Retrieval with Attentive Deep Local Features (DELF), в которой был описан подход к построению локальных признаков на основе сверточных нейросетей с механизмом внимания для отбора ключевых точек [32]. В 2020 году Google опубликовал статью Unifying Deep Local and Global Features for Image Search (DELG), ключевым отличием стало использование и обучение одной модели, которая могла бы извлекать как глобальные, так и локальные признаки, эта идея улучшила результаты и ускорила вычисления эмбедингов [24]. Двухэтапная система является ключевым отличием от решения Яндекса.

Сервис Google Images, аналогично Яндекс Картинкам, обладает возможностью фильтрации контента — через функцию SafeSearch, добавляя в пайплайн этап постобработки [14]. Также стоит отметить, что у сервиса Google имеется два уровня поиска похожих изображений: поиск совпадений и поиск точных совпадений, что является конкурентным преимуществом.

Схематичные устройства архитектур DELF и DELG представлены на рисунках 1.1 и 1.2 соответственно.

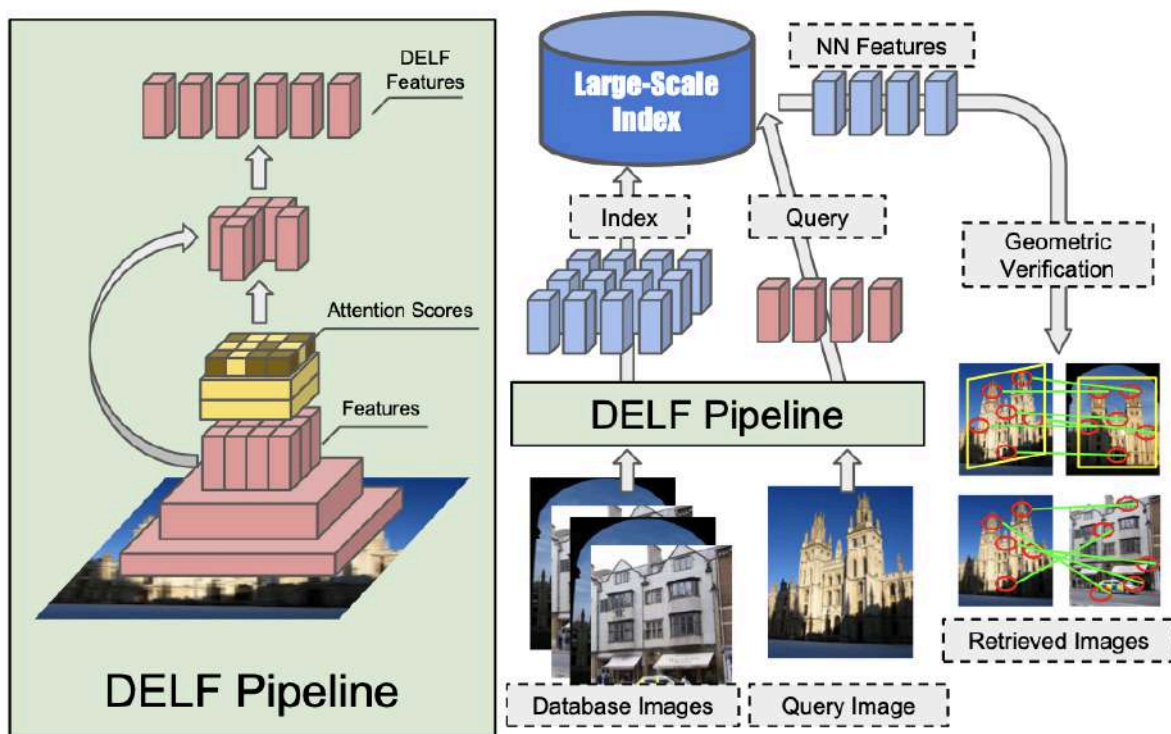


Рисунок 1.1. Официальная схема архитектуры DELF

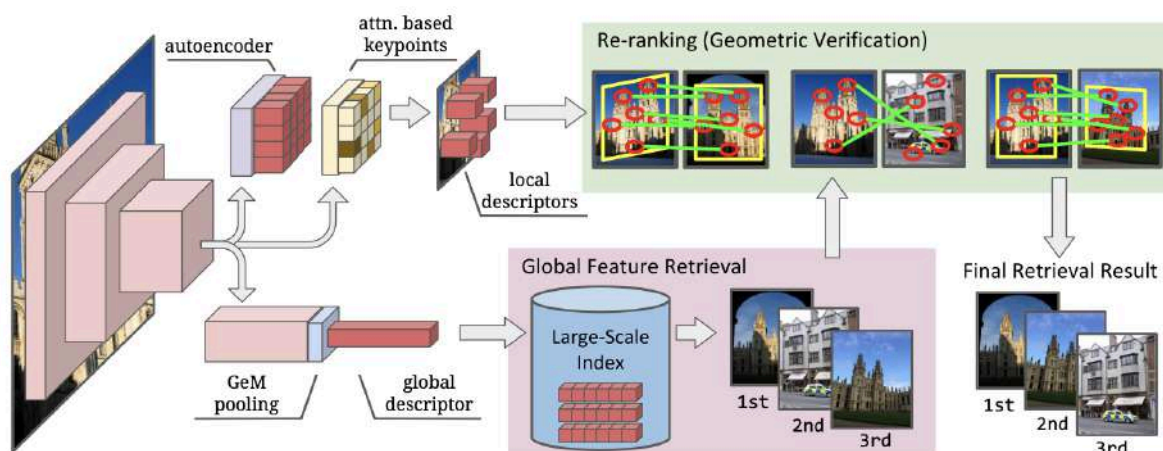


Рисунок 1.2. Официальная схема архитектуры DELG

Подход к поиску схожих изображений в Bing Images аналогичен решениям Яндекс и Google. В материалах Microsoft прямо описывается применение каскадного машинного обучения ранжированию для работы с десятками миллиардов изображений при сохранении задержки ответа в

пределах 200 миллисекунд [28]. На рисунке 1.3 представлен обзор рабочего процесса системы визуального поиска в Bing Images.

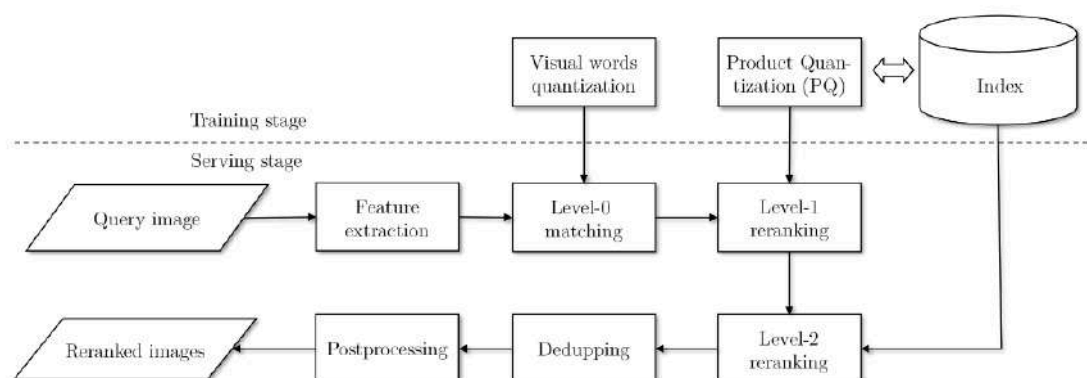


Рисунок 1.3. Трехуровневая каскадная система ранкинга в Bing Images

Ключевое отличие Bing заключается в более формализованном и открытом подходе к архитектуре поиска, отраженном в его публичных API. Система продуктово разделена на два слоя: общий поиск изображений (Bing Image Search) и углубленный анализ по конкретной картинке (Bing Visual Search) [2-3]. Процесс взаимодействия построен на использовании специальных токенов `imageInsightsToken`, которые возвращаются вместе с результатами общего поиска [17]. Эти токены затем могут быть использованы для запуска целевого запроса на поиск похожих изображений или получение дополнительной информации, что делает конвейер обработки более структурированным по сравнению с конкурентами.

Bing Images поддерживает поиск на уровне объектов, а не только целых изображений, что характерно и для других крупных систем визуального поиска [9]. Как и другие глобальные поисковики, Bing включает механизм фильтрации контента, что отражено в параметре `safeSearch` [13]. Заклячая, решение от Microsoft, разделяя общие принципы с конкурентами, отличается более прозрачной и структурированной API-моделью.

Локальные системы управления фотоархивами решают задачу поиска похожих изображений в более замкнутой постановке по сравнению с глобальными поисковиками: поиск выполняется по персональной коллекции, а ключевым становится этап индексации, после которого запросы сводятся к сравнению заранее вычисленных представлений изображений. Общий пайплайн выделяется так: загрузка или импорт изображений в локальное хранилище, фоновая обработка и построение индексов, поиск похожих изображений как операции k-ближайших соседей или пороговой фильтрации по мере сходства, рассчитанной на этих признаках.

Immich реализует нейросетевую составляющую CBIR через фоновое построение эмбедингов и хранение их в базе данных. После загрузки медиа система запускает фоновые задачи в jobs и workers, которые последовательно извлекают метаданные и выполняют иные задачи [7]. В документации Immich указано, что он использует нейросетевые CLIP модели, а векторный поиск реализован на стороне Postgres через расширение VectorChord, то есть эмбединги хранятся и используются для поиска непосредственно внутри СУБД [18]. В рамках поиска похожих изображений ключевой практический режим в Immich это обнаружение дублей и близких изображений по порогу близости Maximum detection distance в векторном пространстве: система использует эмбединги, полученные CLIP, для нахождения вероятных дублей и позволяет настраивать чувствительность, фактически задавая порог на сходство [4]. Таким образом, конвейер Immich в части CBIR можно описать следующим образом: изображение сохраняется в хранилище, далее в фоне вычисляется эмбединг и сохраняется в базе, после чего поиск похожих изображений выполняется как запрос к векторному хранилищу эмбедингов с возвращением ближайших соседей и элементов, удовлетворяющих порогу близости.

PhotoPrism реализует поиск похожих изображений более гибридно, сочетая точное обнаружение дублей на уровне файлов, приближённые признаки визуального сходства и нейросетевой подход. Библиотека изображений хранится в каталоге `originals`, который индексируется при добавлении новых файлов [10]. Для дубликатов PhotoPrism использует файловые признаки, отмечается, что во время индексации дубликаты автоматически обнаруживаются и пропускаются, а сравнение выполняется на основе SHA1-контрольной суммы, специального криптографического алгоритма, переводящего входные данные любого размера в уникальную 160-битную (40 шестнадцатеричных символов) строку, и размера файла [5]. Такой подход соответствует строгому критерию точного дубля и не покрывает случаи, когда изображение изменено сжатием, кроем или иными методами. Для визуальной близости PhotoPrism вычисляет перцептивный хэш, который выступает отпечатком изображения и может использоваться для сортировки результатов по визуальной схожести [11]. Математически это можно описать как отображение исходного изображения X в битовую строку $h(X)$ фиксированного размера 24 бит $h: X \rightarrow \{0, 1\}^{24}$ и сравнения изображений Q и X по расстоянию Хэмминга:

$$dH(h(q), h(x)) = \sum_{j=1}^m I(h_j(q) \neq h_j(x)),$$

где I — индикатор [16]. Чем меньше dH , тем визуально ближе изображения. Наиболее близкой к классическому CBIR в PhotoPrism является подсистема распознавания лиц, поскольку она реализует сравнение объектов по эмбедингам. После детекции лиц PhotoPrism генерирует 512-мерные векторные представления с использованием FaceNet, далее эти векторы используются для сопоставления лиц на разных фотографиях и для кластеризации похожих лиц алгоритмом DBSCAN с последующим ручным подтверждением привязки к человеку [6]. В такой

постановке лицо на изображении выступает как объект запроса, а поиск похожих сводится к сравнению эмбедингов лиц и их группировке.

DigiKam, как настольное приложение, реализует CBIR через построение базы визуальных отпечатков изображений и сравнения этих отпечатков при поиске дублей и похожих изображений. Отпечатки — компактные числовые представления изображений, в документации прямо отмечено, что они строятся на основе Вейвлетов и алгоритма Хаара, а вычисления выполняются автоматически в фоновом режиме [19]. После вычисления отпечатков всей базы изображений пользователь может запустить поиск дублей с заданием диапазона Similarity Range в процентах, который задаёт порог близости изображений для выдачи [8]. В итоге DigiKam реализует классический локальный CBIR пайплайн: офлайн построение отпечатков для базы и онлайн сравнение отпечатков с пороговой фильтрацией и ранжированием по степени сходства.

В целом локальные решения демонстрируют общую архитектурную идею CBIR: перенос вычислительно тяжёлой части построения представлений изображений в индексацию и выполнение поиска похожих изображений как операции сравнения представлений с использованием top-k или пороговой фильтрации. При этом Immich ближе к современной нейросетевой постановке CBIR, поскольку использует эмбединги и векторный поиск по базе данных, а PhotoPrism и DigiKam, напротив, во многом опираются на отпечатки и хэши. Можно заметить, что все эти системы являются одношаговыми.

1.2. Обзор метрик ранжирования и методов сравнения объектов в векторном пространстве

При решении задачи визуального поиска изображений недостаточно только определить способ представления изображений в векторном пространстве и меру близости между ними. Не менее важной является

задача оценки качества ранжированной выдачи, то есть того, насколько высоко в списке найденных результатов располагаются действительно релевантные изображения. В данной работе релевантность задаётся бинарно: изображение из базы считается релевантным изображению-запросу, если оно принадлежит к тому же классу, что и запрос, и нерелевантным в противном случае. Такая постановка позволяет использовать классические метрики ранжирования для top-k retrieval и оценки качества упорядоченной выдачи. В качестве основных метрик в работе выбраны Precision@k, Recall@k, Average Precision, Mean Average Precision, Hit Rate@k и Precision@1. Данный набор позволяет оценить как качество верхней части выдачи, так и качество ранжирования релевантных результатов в целом [22].

Пусть для запроса q система возвращает упорядоченный список изображений, а $rel_q(i)$ — бинарная функция релевантности объекта на позиции i , принимающая значение 1, если объект релевантен запросу, и 0 в противном случае. Обозначим через R_q общее число релевантных изображений в базе для запроса q , а через k глубину рассматриваемой части выдачи.

Метрика Precision@k показывает, какая доля изображений среди первых k результатов является релевантной. Она определяется следующим образом:

$$P@k(q) = (1 / k) * \sum_{i=1}^k rel_q(i).$$

Содержательно Precision@k характеризует точность верхней части выдачи. Чем выше значение этой метрики, тем больше релевантных изображений содержится среди первых позиций.

Метрика Recall@k показывает, какая доля всех релевантных изображений была найдена системой среди первых k результатов:

$$R@k(q) = (1 / R_q) * \sum_{i=1}^k rel_q(i).$$

Если Precision@k отражает точность в верхней части выдачи, то Recall@k характеризует её полноту.

Частным случаем метрики Precision@k является Precision@1:

$$P@1 = rel_q(1).$$

При усреднении по множеству запросов данная метрика показывает, насколько часто самый первый найденный результат относится к тому же классу, что и изображение-запрос. С практической точки зрения Precision@1 является одной из наиболее наглядных характеристик качества системы, поскольку напрямую отражает корректность лучшего ответа.

Метрики Precision@k и Recall@k оценивают только фиксированную верхнюю часть списка, однако не в полной мере учитывают расположение всех релевантных объектов в ранжированной выдаче. Для более полной оценки используется Average Precision. Для одного запроса Average Precision определяется как среднее значение precision на тех позициях, где встречаются релевантные объекты:

$$AP(q) = (1 / R_q) * \sum_{i=1}^N P@i(q) * rel_q(i),$$

где N — длина ранжированного списка. Таким образом, вклад в значение Average Precision дают только позиции, на которых расположены релевантные изображения, причём более высокое положение релевантного результата в выдаче увеличивает итоговое значение метрики. Это делает Average Precision удобной характеристикой качества ранжирования для одного запроса.

Для оценки качества системы на множестве запросов используется Mean Average Precision, представляющая собой среднее значение Average Precision по всем запросам тестовой выборки:

$$mAP = (1 / |Q|) * \sum_{q \in Q} AP(q),$$

где Q — множество запросов. Метрика mAP удобна тем, что даёт единый итоговый показатель качества retrieval-системы и одновременно учитывает порядок расположения релевантных изображений для каждого запроса.

Дополнительно в работе используется метрика $HitRate@k$. Для отдельного запроса она принимает значение 1, если среди первых k результатов присутствует хотя бы одно релевантное изображение, и 0 в противном случае. Итоговое значение вычисляется усреднением по всем запросам:

$$HitRate@k = (1 / |Q|) * \sum_{q \in Q} HitRate@k(q).$$

Содержательно $Hit Rate@k$ показывает, для какой доли запросов система сумела хотя бы один раз попасть в релевантный класс в пределах первых k позиций. Эта метрика является более грубой, чем $Precision@k$ или AP , однако хорошо дополняет их как практическая $top-k$ -характеристика.

Выбор именно этих метрик обусловлен особенностями постановки задачи. Поскольку релевантность в работе бинарная, использование более сложных метрик, ориентированных на градуированную релевантность, не является необходимым. $Precision@k$, $Recall@k$ и $Precision@1$ позволяют оценить качество верхней части выдачи, $Hit Rate@k$ — факт успешного попадания в $top-k$, а $Average Precision$ и $Mean Average Precision$ — качество ранжирования релевантных изображений в целом. В совокупности такой набор метрик даёт достаточно полное и интерпретируемое представление о качестве разрабатываемой системы визуального поиска.

После преобразования изображения нейронной сетью в компактное векторное представление задача визуального поиска может быть сведена к сравнению объектов в векторном пространстве. В такой постановке

каждому изображению сопоставляется эмбединг фиксированной размерности, а поиск похожих изображений заключается в вычислении меры близости между вектором запроса и векторами изображений базы. Подобная постановка является естественной для современных retrieval-систем, поскольку позволяет перейти от сравнения исходных пиксельных представлений к сравнению более информативных признаков, отражающих содержательные свойства изображения. В качестве основных методов сравнения в данной работе рассматриваются косинусное сходство, скалярное произведение и евклидово расстояние. Выбор именно этих методов обусловлен тем, что они являются базовыми и наиболее распространёнными способами сравнения вещественных векторов в задачах поиска и ранжирования [23].

Наиболее естественной мерой сходства для эмбедингов является косинусное сходство. Оно характеризует близость направлений двух векторов и тем самым показывает, насколько сходны их относительные структуры независимо от абсолютной длины. В литературе по информационному поиску косинусное сходство рассматривается как стандартный способ количественно оценивать близость объектов в векторном пространстве. Для задачи визуального поиска это особенно удобно, поскольку длина эмбединга сама по себе часто менее информативна, чем его направление в пространстве признаков. Косинусное сходство двух векторов x и y определяется по формуле:

$$\text{sim}(x, y) = \frac{x \cdot y}{\|x\| * \|y\|},$$

где $x \cdot y$ — скалярное произведение векторов, а $\|x\|$ и $\|y\|$ — их евклидовы нормы. Чем ближе значение этой величины к единице, тем более похожими считаются сравниваемые объекты.

С косинусным сходством тесно связано скалярное произведение. Оно вычисляется как сумма попарных произведений координат:

$$x \bullet y = \sum_{i=1}^d x_i y_i,$$

где d — размерность векторного пространства. Если векторы заранее нормированы по длине, то скалярное произведение и косинусное сходство дают одинаковое ранжирование объектов, поскольку в этом случае $\|x\| = \|y\| = 1$, и формула косинусного сходства упрощается до:

$$\text{sim}(x, y) = x \bullet y.$$

В практическом смысле это делает скалярное произведение удобным методом сравнения, поскольку оно вычисляется просто и часто используется как базовая операция при поиске по эмбедингам. Однако для ненормализованных векторов скалярное произведение начинает зависеть не только от направления, но и от длины вектора, поэтому его поведение может отличаться от косинусного сходства.

Другим классическим способом сравнения является евклидово расстояние. В отличие от косинусного сходства, оно измеряет не сходство направлений, а геометрическое расстояние между точками в пространстве признаков. Евклидово расстояние между векторами x и y задаётся формулой:

$$\text{dist}(x, y) = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}.$$

Также евклидово расстояние между векторами x и y можно задать следующей формулой:

$$\|x - y\|^2 = 2 - 2(x \bullet y),$$

если $\|x\| = \|y\| = 1$. Это означает, что для векторных представления с единичной нормой максимизация косинусного сходства эквивалентна минимизации евклидова расстояния, и оба подхода приводят к одинаковому порядку ранжирования.

После выбора меры близости поиск релевантных изображений может быть описан как задача поиска ближайших соседей. Для изображения-запроса вычисляется его векторное представление, после чего среди объектов базы выбираются те, чьи векторы наиболее близки к нему по выбранной мере. В простейшем случае результатом является список из k ближайших объектов, упорядоченных по убыванию сходства или по возрастанию расстояния. Формально при использовании меры сходства это можно записать как выбор множества $TopK(q) = \arg \max_k sim(x, q)$, где q — вектор запроса, D — база изображений. Если же используется мера расстояния, то задача принимает вид $TopK(q) = \arg \min_k dist(x, q)$.

Подводя итоги подглавы, для данной работы методы сравнения объектов в векторном пространстве целесообразно описывать через три базовые меры: косинусное сходство, скалярное произведение и евклидово расстояние, а сам поиск похожих изображений трактовать как поиск ближайших соседей по эмбедингам.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ ОЦЕНКИ СХОДСТВА ИЗОБРАЖЕНИЙ

В данной главе рассматриваются основные архитектурные подходы к построению нейросетевых моделей, пригодных для генерации векторных представлений изображений в задаче визуального поиска. Рассмотрены следующие классы: сверточные нейронные сети, трансформерные архитектуры компьютерного зрения и модели семейства CLIP. Выбор именно этих направлений обусловлен тем, что они отражают основные этапы развития современных методов извлечения визуальных признаков.

Вторая подглава посвящена требованиям к системе оценки сходства изображений и структуре двухэтапного поиска похожих кандидатов.

2.1. Анализ архитектур нейронных сетей для генерации векторных представлений

Сверточные нейронные сети являются первым крупным классом нейросетевых архитектур, который обеспечил практический переход от ручных визуальных признаков к автоматически обучаемым представлениям изображений. Исторически одной из наиболее ранних и известных архитектур данного семейства является LeNet-5, предложенная Яном Лекуном и соавторами в работе *Gradient-Based Learning Applied to Document Recognition* в 1998 году [31]. Данная модель была ориентирована на задачу распознавания рукописных символов и продемонстрировала, что операция свертки, совместно с обучаемыми фильтрами и иерархией признаков, позволяет эффективно извлекать устойчивые визуальные представления. В дальнейшем развитие CNN-архитектур на некоторое время замедлилось, однако резкий рост интереса к ним произошел после выхода работы *ImageNet Classification with Deep Convolutional Neural Networks* в 2012 году, в работе была предложена архитектура AlexNet [30]. В ней авторы показали, что глубокая сверточная сеть, обученная на

крупномасштабном датасете ImageNet, способна существенно превзойти предшествующие методы и обеспечить прорывной результат в соревновании ILSVRC 2012. Именно AlexNet принято рассматривать как отправную точку современного этапа развития глубоких CNN в задачах компьютерного зрения.

Дальнейшее развитие сверточных архитектур в значительной степени связано с тремя моделями, которые целесообразно подробно рассмотреть в рамках данной работы. Архитектура VGG показала, что повышение глубины сети при использовании малых сверток размером 3×3 может приводить к существенному росту качества распознавания, а сети глубиной 16-19 весовых слоев оказались конкурентоспособными на ImageNet [34]. Архитектура GoogLeNet, представленная в работе *Going Deeper with Convolutions*, предложила модуль Inception и использование вспомогательных голов при обучении, что позволило увеличить глубину и ширину сети без пропорционального роста вычислительной стоимости, итоговая реализация данной идеи имела глубину в 22 слоя [35]. Наконец, архитектура ResNet ввела принцип residual learning, позволивший существенно упростить обучение очень глубоких сетей: в исходной работе авторы рассматривали сети глубиной до 152 слоев и показали, что residual-подход позволяет увеличить глубину модели без характерной для предшествующих архитектур деградации качества [27]. По этой причине именно ResNet целесообразно рассматривать как основной CNN-бейзлайн в дальнейшей части работы, тогда как VGG и GoogLeNet позволяют последовательно показать эволюцию идей увеличения глубины и повышения вычислительной эффективности сверточных сетей.

Общий принцип работы сверточной нейронной сети заключается в последовательном преобразовании исходного изображения в набор все более абстрактных признаков. Если на вход сети подается изображение в виде многомерного массива пиксельных значений, то первые слои

извлекают простейшие локальные структуры, такие как границы, контрастные переходы и простые текстуры, а по мере увеличения глубины сети признаки становятся более сложными и начинают соответствовать уже не отдельным локальным паттернам, а более содержательным элементам изображения. Именно такая иерархическая организация представлений лежит в основе CNN-архитектур, начиная с LeNet и заканчивая более поздними моделями семейства VGG, GoogLeNet и ResNet.

Ключевой операцией в сверточной сети является свертка. Её смысл состоит в том, что небольшой обучаемый фильтр последовательно перемещается по изображению или по карте признаков предыдущего слоя и на каждой позиции вычисляет взвешенную сумму значений в локальном окне. Если обозначить входную карту признаков как X , ядро свертки размера $m \times n$ как K , а выходную карту как Y , то значение выходной карты в позиции (i, j) выходного канала k вычисляется как:

$$Y_k(i, j) = \sum_{c=1}^C \sum_{u=0}^{m-1} \sum_{v=0}^{n-1} X_c(i+u, j+v) \cdot K_{k,c}(u, v) + b_k,$$

где b_k — смещение, а c и C — индекс входного канала и число входных каналов соответственно. В результате применения набора таких фильтров формируется несколько новых карт признаков, каждая из которых реагирует на определенный тип визуального паттерна. Важной особенностью сверточных слоев является разделение весов: один и тот же фильтр применяется ко всем пространственным позициям изображения. Благодаря этому число параметров оказывается существенно меньше, чем в полносвязных слоях, а сама сеть получает свойство чувствительности к локальным структурам при относительной устойчивости к их смещению в пределах изображения.

После операции свертки обычно применяется нелинейная функция активации. Она необходима для того, чтобы сеть не сводилась к

последовательности линейных преобразований. Если бы между слоями отсутствовала нелинейность, то даже глубокая модель в функциональном смысле была бы эквивалентна одному линейному преобразованию и не могла бы описывать сложные зависимости в данных. В современных CNN наиболее распространенной функцией активации является ReLU, задаваемая выражением:

$$f(x) = \max(0, x).$$

Её практическое значение заключается в простоте вычисления и в том, что она облегчает обучение глубоких сетей по сравнению с более ранними нелинейностями, например сигмойдой или гиперболическим тангенсом.

Другим важным элементом CNN является пуллинг. Его основная задача состоит в уменьшении пространственного разрешения карт признаков при сохранении наиболее значимой информации. К примеру можно использовать max-pooling, при котором для каждого локального окна выбирается максимальное значение. Если обозначить локальное окно признаков как $\Omega(i, j)$, то max-pooling записывается следующим образом:

$$P(i, j) = \max_{(u, v) \in \Omega(i, j)} X(u, v).$$

Использование пуллинга позволяет уменьшить размерность промежуточных представлений, сократить вычислительные затраты и сделать признаки менее чувствительными к небольшим локальным сдвигам и деформациям изображения. Таким образом, типичный сверточный блок состоит из свертки, нелинейной активации и, во многих случаях, операции понижения пространственного разрешения. Многократное повторение таких блоков позволяет постепенно переходить от низкоуровневых признаков к высокоуровневым.

На завершающих этапах обработки сверточная сеть преобразует полученные карты признаков в компактное векторное представление, которое затем может использоваться либо для классификации, либо как эмбединг изображения в задаче визуального поиска.

Архитектура VGG была предложена Кареном Симоньяном и Эндрю Зиссерманом в работе Very Deep Convolutional Networks for Large-Scale Image Recognition и стала одной из ключевых моделей в развитии глубоких сверточных сетей [34]. Основная идея архитектуры заключается в систематическом увеличении глубины сети при сохранении максимально простой и однородной структуры. В отличие от более ранних архитектур, в которых использовались свертки различных размеров и более сложные инженерные решения, в VGG почти вся сеть строится из повторяющихся блоков, состоящих из сверток малого размера 3×3 , функций активации ReLU и периодического max-pooling. Такой подход позволил показать, что рост глубины сам по себе, при достаточно аккуратной организации архитектуры, способен существенно улучшать качество извлечения визуальных признаков.

Ключевым отличием VGG от предшествующих моделей стало использование малых сверток в большом количестве. Если обозначить число входных и выходных каналов через C , то одна свертка размером 5×5 содержит $25C^2$ обучаемых весов, тогда как две последовательные свертки размером 3×3 содержат только $18C^2$ параметров. При этом эффективное рецептивное поле двух сверток 3×3 соответствует одной свертке 5×5 , однако между ними появляется дополнительная нелинейность. Аналогично три свертки 3×3 дают рецептивное поле, сопоставимое со сверткой 7×7 , но с меньшим числом параметров и большей выразительностью модели. Именно эта идея и лежит в основе архитектуры VGG: вместо использования крупных ядер сеть строится как глубокая последовательность сравнительно простых локальных преобразований. Под рецептивным полем понимается область исходного изображения, значения которой влияют на значение данного элемента карты признаков.

В работе были рассмотрены несколько конфигураций сети, различающихся глубиной, среди которых наибольшую известность

получили VGG-16 и VGG-19. В этих моделях сверточная часть организована в виде нескольких блоков, внутри которых пространственное разрешение сохраняется, а число каналов постепенно увеличивается. После каждого такого блока применяется max-pooling, уменьшающий размер карты признаков. На завершающем этапе после сверточной части следуют полносвязные слои, формирующие итоговое представление, используемое для классификации.

На рисунках 2.1 и 2.2 представлены основные конфигурации VGG и схематичное устройство VGG-16 соответственно.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Рисунок 2.1. Основные конфигурации архитектуры VGG

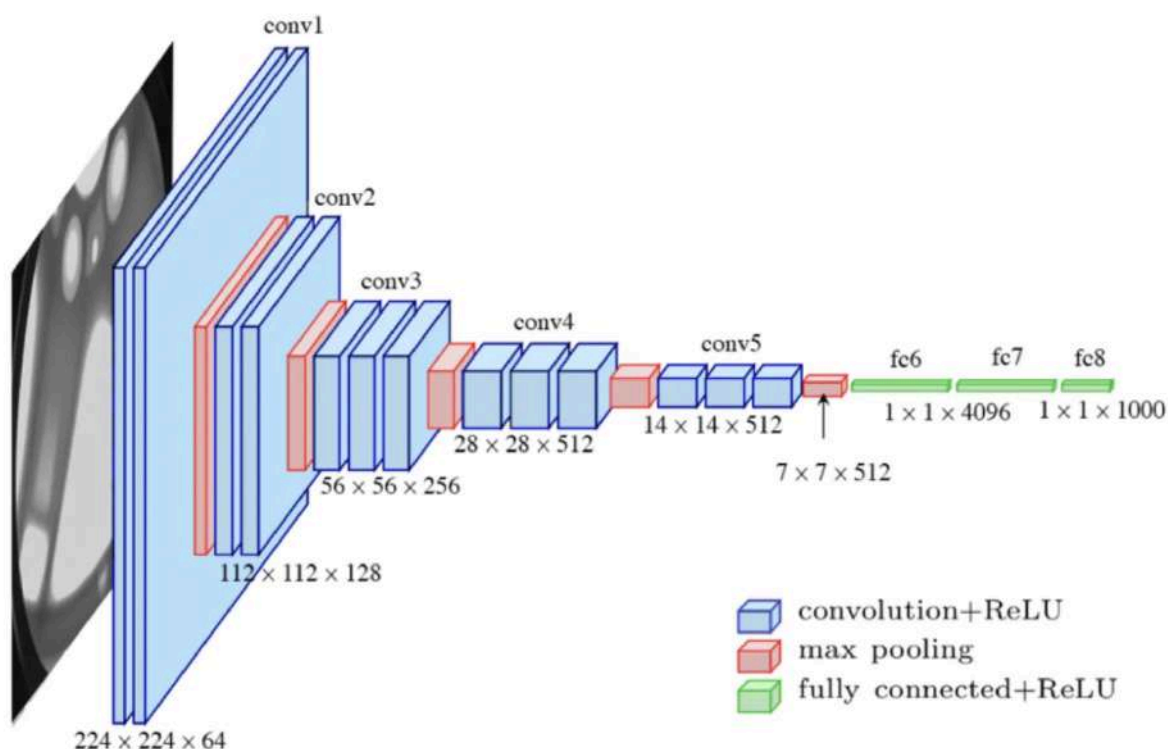


Рисунок 2.2. Структура архитектуры VGG-16

С точки зрения данной работы архитектура VGG представляет интерес как важный этап развития CNN-подхода к извлечению признаков изображений. Она наглядно показывает, что даже без сложных специальных блоков сверточная сеть может формировать сильные визуальные представления, если достаточно глубока и построена на повторяющихся локальных операциях. В то же время существенным недостатком VGG является большое число параметров, особенно в полносвязной части, что делает модель вычислительно тяжелой по сравнению с более поздними архитектурами.

Архитектура GoogLeNet была предложена Кристианом Сегеди и соавторами в работе *Going Deeper with Convolutions* и стала следующим важным этапом развития сверточных нейронных сетей после моделей, в которых основной акцент делался прежде всего на росте глубины сети [35]. Если архитектура VGG демонстрировала эффективность глубокой и однородной последовательности сверток малого размера, то в GoogLeNet

основной акцент был сделан на более эффективную организацию вычислений внутри слоя. Целью авторов стало увеличение глубины и выразительности сети без резкого роста числа параметров и вычислительной стоимости. Именно поэтому архитектура GoogLeNet представляет особый интерес как пример перехода от простого наращивания глубины к более сложному проектированию внутренних блоков сети.

Ключевой идеей GoogLeNet является Inception-модуль. В отличие от обычного сверточного блока, в котором входной тензор проходит через одну последовательность преобразований, Inception-модуль обрабатывает вход параллельно по нескольким ветвям. В таких ветвях используются свертки 1×1 , 3×3 , 5×5 , а также операция pooling. После этого результаты всех ветвей объединяются по каналному измерению. Такая конструкция позволяет одновременно извлекать признаки разного масштаба: свертки малого размера лучше реагируют на локальные детали, более крупные ядра позволяют учитывать более широкий контекст, а pooling добавляет устойчивость к небольшим пространственным смещениям. За счет этого сеть получает возможность анализировать изображение более гибко, чем в случае использования только одного типа свертки на каждом уровне.

На рисунке 2.3 изображены Inception-модули, представленные в работе Going Deeper with Convolutions.

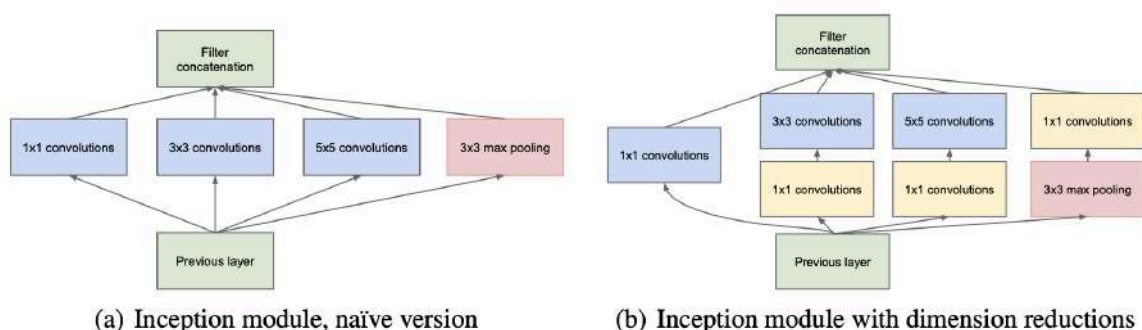


Рисунок 2.3. Inception-модули, представленные командой Google

В итоговой архитектуре использовался блок Inception module with dimension reductions за счет использования свертки 1×1 в качестве понижающей проекции перед более дорогими свертками 3×3 и 5×5 . Если число входных каналов велико, то непосредственное применение крупных сверток требует значительных вычислительных затрат. Введение промежуточной свертки 1×1 позволяет сначала уменьшить число каналов, а затем выполнять более сложную обработку уже в пространстве меньшей размерности. С математической точки зрения такая операция представляет собой линейное преобразование по каналному измерению в каждой пространственной позиции. Пусть в позиции (i, j) входной вектор по каналам равен:

$$x(i, j) = [X_1(i, j), X_2(i, j), \dots, X_C(i, j)] \in R^C.$$

Свертка 1×1 с C' фильтрами вычисляет новый вектор $y(i, j) \in R^{C'}$ по формуле:

$$Y_k(i, j) = \sum_{c=1}^C W_{k,c} X_c(i, j) + b_k; k = 1, \dots, C',$$

где $W_{k,c}$ — обучаемые веса свертки 1×1 , связывающие входной канал c с выходным каналом k , b_k — смещение. Благодаря этому GoogLeNet удается сочетать высокую архитектурную сложность с относительно экономным использованием параметров. Дополнительно в сети были использованы вспомогательные классификаторы, подключаемые к промежуточным слоям. Такие головы участвовали в обучении и частично ослабляли проблему затухания градиента в глубокой архитектуре.

С точки зрения общей организации GoogLeNet представляет собой глубокую сеть, состоящую из последовательности Inception-модулей, объединенных с начальными сверточными слоями и завершающим классификационным блоком. В оригинальной работе рассматривалась сеть глубиной 22 слоя по числу параметризованных слоев, при этом число

параметров оказалось заметно меньше, чем у более простых по структуре, но тяжелых сетей предыдущего поколения. Для данной работы архитектура GoogLeNet представляет интерес как модель, в которой задача извлечения визуальных признаков решается не только за счет увеличения глубины, но и за счет многошкальной обработки и более эффективного распределения вычислений внутри сети.

На рисунках 2.4 и 2.5 представлена архитектура GoogLeNet.

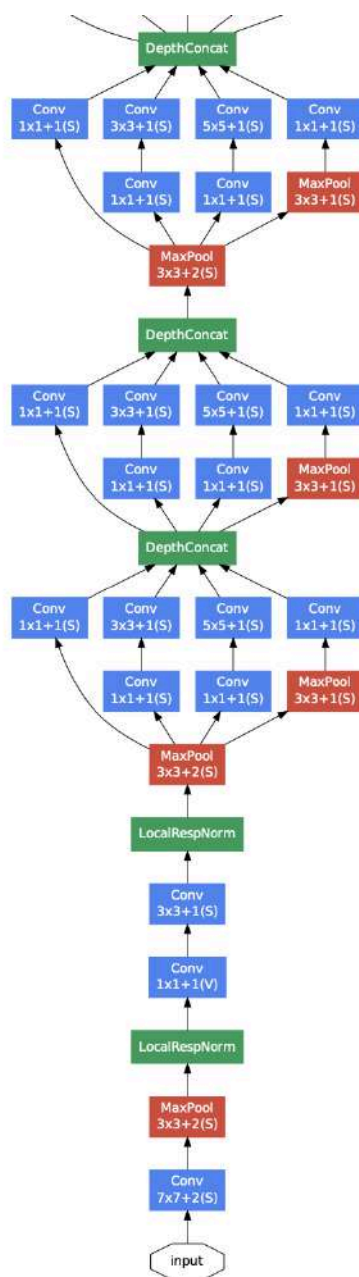


Рисунок 2.4. GoogLeNet (Начало) с Inception-модулями

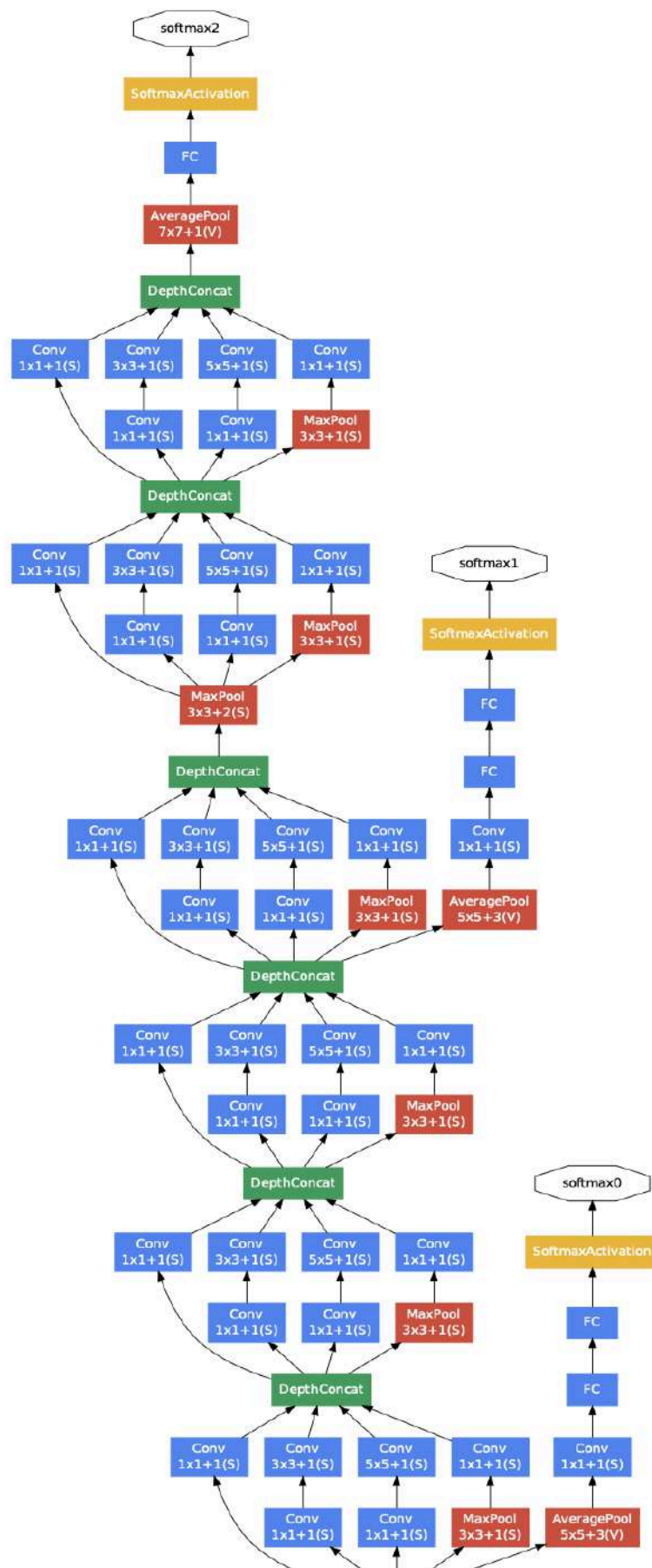


Рисунок 2.5. GoogLeNet (Конец) и вспомогательные классификаторы

Архитектура ResNet была предложена Хэ Каймином и соавторами в работе Deep Residual Learning for Image Recognition и остаётся бейзлайном в мире компьютерного зрения и по сей день [27]. Если в архитектурах VGG и GoogLeNet основное внимание уделялось либо увеличению глубины сети, либо более эффективной организации вычислений внутри отдельных модулей, то в ResNet ключевая проблема была поставлена иначе. Авторы обратили внимание на то, что при дальнейшем увеличении глубины сети качество начинает не улучшаться, а падать, причем речь идет не только о переобучении, но и о трудностях самой оптимизации глубокой модели. В результате была предложена архитектура, позволяющая обучать существенно более глубокие сети за счет введения остаточного обучения.

Основной идеей ResNet является замена прямого приближения некоторого отображения $H(x)$ на приближение остаточной функции $F(x)$, связанной с исходным отображением так:

$$H(x) = F(x) + x,$$

где x — вход блока, $F(x)$ — результат последовательности сверточных преобразований внутри блока. Такой подход называется residual-подходом: вместо того чтобы обучать слой непосредственно строить полное новое представление, сеть обучается вычислять лишь приращение относительно входа. После этого входной сигнал x добавляется к выходу сверточной части блока по skip connection. Выход остаточного блока определяется так:

$$y = F(x, \{W_i\}) + x,$$

где $\{W_i\}$ — набор обучаемых параметров внутри блока. Таким образом skip connections послужили решением проблеме затухания градиентов и позволили им беспрепятственно распространяться по сети при обратном распространении ошибки. В ResNet появилась возможность не только извлекать новые признаки, но и сохранять уже полезные представления, если их дальнейшее существенное изменение не требуется.

В исходной работе рассматривались различные конфигурации сети, такие как ResNet-18, ResNet-34, ResNet-50, ResNet-101 и ResNet-152. На рисунке 2.6 представлена структура модели ResNet-18, как самой простой для визуального представления среди семейства ResNet.

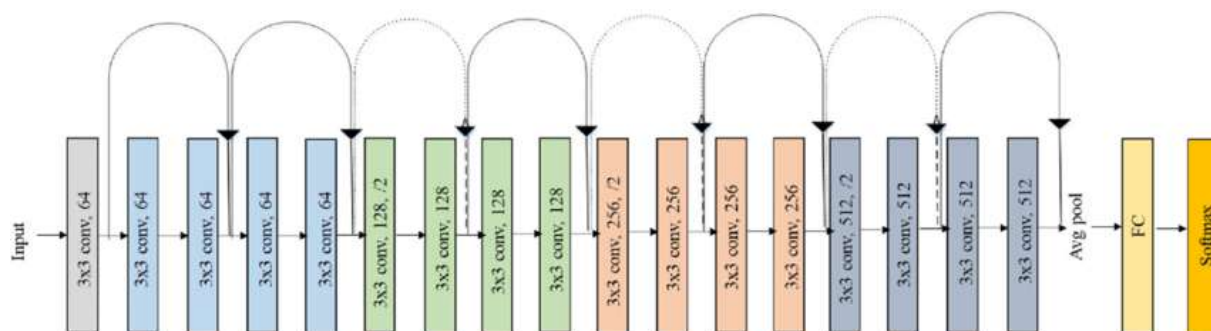


Рисунок 2.6. Архитектура ResNet-18

ResNet сочетает сравнительную концептуальную простоту с высокой практической эффективностью. По этой причине именно ResNet целесообразно рассматривать как основной сверточный бейзлайн для дальнейшей работы, тогда как VGG и GoogLeNet играют роль архитектур, демонстрирующих предыдущие этапы развития CNN-подхода. Кроме того residual-подход делает сеть удобной как для обучения с нуля, так и для использования предобученных весов, что особенно важно в экспериментальной части ВКР.

Следующим важным этапом развития архитектур для извлечения визуальных признаков стало появление Vision Transformer, или ViT. Данная архитектура была предложена Алексеем Досовицким и соавторами в работе An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale, впервые опубликованной в 2020 году и представленной на ICLR 2021 [25]. Значимость этой работы состоит в том, что авторы показали возможность применения трансформера к задаче распознавания изображений без использования сверточных блоков как основы модели. Если для CNN характерно последовательное извлечение локальных

признаков при помощи сверток и постепенное расширение рецептивного поля, то в ViT изображение изначально преобразуется в последовательность патчей, после чего обрабатывается так же, как последовательность токенов в трансформерах для обработки текста.

Основная идея ViT заключается в том, что изображение рассматривается не как двумерная карта, над которой необходимо выполнять свертки, а как упорядоченный набор непересекающихся фрагментов фиксированного размера. Пусть входное изображение задается тензором $x \in R^{H \times W \times C}$, где H , W и C — высота, ширина и число каналов изображения соответственно. Тогда изображение разбивается на патчи размера $P \times P$, и число таких патчей равно:

$$N = \frac{HW}{P^2}.$$

Каждый патч разворачивается в вектор размерности $P^2 C$, после чего для него вычисляется линейное вложение в латентное пространство признаков размерности D . Если обозначить i -й патч через x_p^i , то его представление после линейной проекции можно записать как:

$$z_i = x_p^i E,$$

где $x_p^i \in R^{1 \times (P^2 C)}$, $E \in R^{(P^2 C) \times D}$, а E — обучаемая матрица проекции. Таким образом, уже на первом этапе модель переводит изображение в последовательность векторов одинаковой размерности, пригодную для дальнейшей обработки трансформером.

Поскольку стандартный трансформер сам по себе не содержит информации о порядке элементов последовательности, в ViT к эмбедингам патчей добавляются позиционные эмбединги. Кроме того, в начало последовательности добавляется специальный обучаемый токен класса, аналогичный токenu [CLS] в NLP-моделях. Входная последовательность первого слоя имеет вид:

$$z_0 = \left[x_{class}; x_p^1 E; x_p^2 E; \dots; x_p^N E \right] + E_{pos},$$

где x_{class} — специальный обучаемый токен [CLS], состояние которого на выходе из сети служит агрегированным представлением всего изображения, а E_{pos} — матрица позиционных эмбеддингов. Благодаря этому модель получает не только содержательную информацию о каждом патче, но и сведения о его положении в изображении.

Дальнейшая обработка выполняется стандартным стеком Transformer Encoder, состоящим из чередующихся блоков multi-head self-attention и полносвязной MLP-части, между которыми используются layer normalization и skip connections. Для l -го слоя преобразования записываются следующим образом:

$$z_l' = MSA(LN(z_{l-1})) + z_{l-1}, z_l = MLP(LN(z_l')) + z_l',$$

где LN обозначает layer normalization, MSA — multi-head self-attention, а MLP — двухслойный полносвязный блок с нелинейностью. Данная схема показывает, что ViT строится как последовательность однотипных энкодерных блоков, в которых каждый токен может взаимодействовать со всеми остальными токенами последовательности.

Ключевым математическим механизмом внутри ViT является self-attention. Для входной матрицы токенов Z сначала вычисляются матрицы запросов, ключей и значений:

$$Q = ZW_Q, K = ZW_K, V = ZW_V,$$

где W_Q, W_K, W_V — обучаемые матрицы параметров. После этого attention-преобразование определяется формулой:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

где d_k — размерность векторов ключей. Смысл данного выражения состоит в том, что для каждого токена вычисляются веса внимания ко всем

остальным токенам, после чего формируется новое представление как взвешенная сумма значений V . В отличие от свертки, где взаимодействие изначально локально ограничено размером ядра, self-attention позволяет моделировать зависимости между любыми двумя патчами изображения уже в пределах одного слоя.

В многоголовом варианте attention вычисляется параллельно в нескольких подпространствах признаков. Если число голов равно h , то каждая голова вычисляется как:

$$head_i = Attention(Q_i, K_i, V_i),$$

а итоговое многоголовое внимание имеет вид:

$$MSA(Z) = Concat(head_1, ..., head_h)W_o,$$

где W_o — обучаемая матрица объединения результатов отдельных голов.

Такая конструкция позволяет модели одновременно учитывать разные типы зависимостей между частями изображения.

На выходе последнего энкодерного блока используется представление специального токена [CLS], которое затем подается в классификационную голову. Если обозначить итоговую последовательность после L слоев через z_L , то для классификации используется первый элемент этой последовательности:

$$y = LN(z_L^0),$$

где z_L^0 соответствует class token. В контексте работы данное представление особенно важно, поскольку оно может рассматриваться как компактный глобальный эмбединг изображения, пригодный для последующего сравнения в векторном пространстве.

На рисунке 2.7 представлено схематичное устройство ViT моделей.

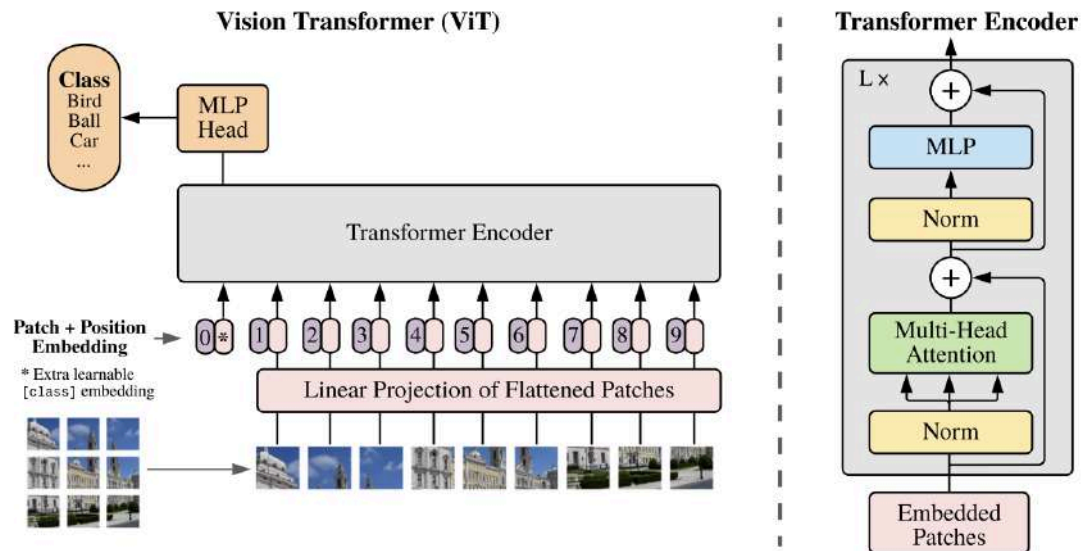


Рисунок 2.7. Структура Vision Transformer

Модель DINO была предложена Матильдой Карон и соавторами в 2021 году в работе *Emerging Properties in Self-Supervised Vision Transformers* [36]. Данный подход стал одним из наиболее заметных результатов в области self-supervised обучения визуальных моделей, поскольку показал, что трансформерные архитектуры способны формировать сильные и содержательные представления изображений даже без использования ручной разметки. Актуальность DINO для задачи визуального поиска обусловлена тем, что получаемые в такой постановке признаки оказываются пригодными не только для глобального сопоставления изображений, но и для более тонкого анализа внутренней структуры изображения, что делает этот подход особенно значимым для CBIR систем с reranked шагом.

В модели одновременно используются две сети одинаковой архитектуры: student и teacher. Обе сети получают разные представления одного и того же изображения, сформированные с помощью случайных аугментаций и кропов, после чего student обучается предсказывать распределение, полученное teacher. Пусть x обозначает изображение, а v —

конкретное преобразование или stop. Тогда выходы student и teacher сетей можно записать как:

$$g_{\theta_s}(v(x)) \text{ и } g_{\theta_t}(v(x)),$$

где θ_s и θ_t — параметры student и teacher соответственно.

Как и в классической дистилляции знаний, обе сети выдают не просто векторы признаков, а вероятностные распределения по выходным компонентам проекционной головы. Если обозначить размерность выходного пространства через K , то для student-сети распределение может быть записано так:

$$P_s^{(i)}(x) = \frac{\exp(g_{\theta_s}(x)^{(i)}/\tau_s)}{\sum_{k=1}^K \exp(g_{\theta_s}(x)^{(k)}/\tau_s)},$$

где τ_s — температура student-сети, а $g_{\theta_s}(x)^{(i)}$ — i -я компонента выхода student-head. Аналогично для teacher-сети используется распределение:

$$P_t^{(i)}(x) = \frac{\exp((g_{\theta_t}(x)^{(i)} - c^{(i)})/\tau_t)}{\sum_{k=1}^K \exp((g_{\theta_t}(x)^{(k)} - c^{(k)})/\tau_t)},$$

где τ_t — температура teacher-сети, а c — центрирующий вектор, вычитаемый из teacher-логитов перед softmax. Уже из этой формулы видно важное отличие DINO от обычного ViT: в DINO вводится специальная схема центрирования и повышение резкости распределения выходов teacher-сети, предназначенная для предотвращения коллапса представлений.

Целевая функция DINO строится как кросс-энтропия между распределениями teacher и student для разных представлений одного и того же изображения. Для одной пары представлений это можно записать следующим образом:

$$L(x) = - \sum_{i=1}^K P_t^{(i)}(x_1) \log P_s^{(i)}(x_2),$$

где x_1 и x_2 — два различных преобразования одного исходного изображения. В общей многовидовой постановке ошибка усредняется по нескольким парам глобальных и локальных вырезов изображения. Тем самым DINO заставляет модель формировать представления, инвариантные к различным преобразованиям изображения, однако в отличие от контрастивных методов не требует явного использования отрицательных пар.

Существенной особенностью DINO является multi-crop training. Student-сеть получает как глобальные, так и локальные crop-представления изображения, тогда как teacher-сеть обрабатывает только глобальные представления. Такая организация обучения побуждает модель сопоставлять локальные фрагменты изображения с его более полным глобальным содержанием. Для ViT это особенно важно, поскольку токены патчей и механизм self-attention позволяют естественным образом связывать локальную и глобальную информацию внутри одного и того же представления. В результате DINO обучает ViT не только формировать общий глобальный эмбединг изображения, но и строить более содержательные локальные признаки, связанные со структурой сцены.

Параметры teacher-сети в DINO не обучаются напрямую градиентным спуском. Вместо этого teacher обновляется как экспоненциальное скользящее среднее параметров student:

$$\theta_t \leftarrow \lambda \theta_t + (1 - \lambda) \theta_s,$$

где λ — коэффициент инерции. Teacher в DINO выступает не как независимая заранее заданная модель, а как сглаженная по времени версия student, задающая более стабильную цель обучения.

Не менее важным компонентом DINO является центрирование teacher-выходов. Центр s обновляется по экспоненциальному скользящему среднему батчевых выходов teacher-сети:

$$c \leftarrow mc + (1 - m) \frac{1}{B} \sum_{i=1}^B g_{\theta_t}(x_i),$$

где m — коэффициент сглаживания, а B — размер батча. Содержательно центрирование препятствует ситуации, при которой одна из координат начинает доминировать во всех предсказаниях teacher. Однако само по себе центрирование может склонять систему к почти равномерному распределению, поэтому в DINO оно используется совместно с повышением резкости распределения, то есть с малой температурой τ_t в softmax teacher-сети.

С точки зрения архитектурного сравнения с классическим ViT особенно важно, что DINO не требует изменения базового механизма patch embedding, encoder-блоков self-attention или MLP. Базовый ViT как backbone сохраняется, однако к нему добавляется projection head, а вся модель обучается в teacher-student режиме без меток. Тем самым DINO отличается от обычного ViT не на уровне структуры отдельного трансформерного блока, а на уровне всей обучающей процедуры и способа формирования представлений. Если в supervised ViT глобальный class token обучается решать задачу классификации по известным классам, то в DINO представления формируются под действием инвариантности к аугментациям и согласования student с teacher без внешней разметки.

Еще одним важным направлением развития моделей извлечения визуальных признаков стала модель CLIP, предложенная Алеком Рэдфордом и соавторами в 2021 году в работе Learning Transferable Visual Models From Natural Language Supervision [33]. В отличие от рассмотренных ранее CNN, ViT и DINO, данная модель изначально строится как мультимодальная система, обучаемая не только на изображениях, но и на парах «изображение — текстовое описание». Актуальность CLIP определяется тем, что такая постановка позволяет

обучать визуальный энкодер на значительно более широком и семантически разнообразном источнике данных, чем обычная разметка фиксированного набора классов. Для задачи визуального поиска это особенно важно, поскольку эмбединги, полученные в подобной постановке, оказываются пригодными не только для классификации, но и для сопоставления изображений по их содержанию в общем векторном пространстве.

Основная идея CLIP заключается в совместном обучении двух энкодеров: визуального и текстового. Визуальный энкодер получает на вход изображение и преобразует его в вектор признаков, а текстовый энкодер получает текстовое описание и также преобразует его в вектор фиксированной размерности. Пусть изображение обозначено через x , текст — через t , тогда выходы двух энкодеров можно записать в виде:

$$u = f_{\theta}(x), v = g_{\phi}(t),$$

где f_{θ} — визуальный энкодер с параметрами θ , а g_{ϕ} — текстовый энкодер с параметрами ϕ . В оригинальной работе в качестве визуального энкодера рассматривались как модифицированные ResNet, так и Vision Transformer, тогда как текстовый энкодер строился на трансформерной архитектуре. Таким образом, CLIP представляет интерес не как отдельный тип visual backbone, а как способ обучения визуальной модели через согласование изображений и естественного языка.

После получения векторов u и v они нормализуются по длине, что позволяет интерпретировать их сравнение через косинусное сходство. Нормализованные представления можно записать следующим образом:

$$\hat{u} = \frac{u}{\|u\|}, \hat{v} = \frac{v}{\|v\|},$$

Для пары изображения i и текста j логит сходства определяется как:

$$s_{ij} = \exp(\tau) \hat{u}_i^T \hat{v}_j,$$

где τ — обучаемый скалярный параметр температуры. Из данной формулы видно, что CLIP обучает изображение и текст попадать в общее пространство признаков, в котором согласованные пары имеют высокое сходство, а несогласованные — низкое. В отличие от DINO, где согласуются различные представления одного и того же изображения, в CLIP согласование происходит между двумя различными модальностями.

Обучение CLIP строится как симметричная контрастивная задача по батчу пар «изображение — текст». Для батча из N согласованных пар каждая строка матрицы сходства должна правильно указывать на соответствующий текст для данного изображения, а каждый столбец — на соответствующее изображение для данного текста. Соответствующие функции потерь можно записать в виде:

$$L_{img} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(s_{ii})}{\sum_{j=1}^N \exp(s_{ij})}, \quad L_{text} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(s_{ii})}{\sum_{j=1}^N \exp(s_{ji})},$$

а общая функция потерь определяется как:

$$L = \frac{1}{2} (L_{img} + L_{text}).$$

Тем самым модель одновременно учится по двум направлениям: находить правильный текст для изображения и правильное изображение для текста. Именно эта симметричная контрастивная постановка и дала название модели — Contrastive Language-Image Pretraining.

С архитектурной точки зрения CLIP отличается от классических supervised-моделей тем, что его целевая функция не привязана к фиксированному числу заранее заданных классов. Если в обычной задаче классификации визуальный энкодер обучается предсказывать один из элементов конечного словаря меток, то в CLIP модель обучается сближать изображение с его текстовым описанием в непрерывном пространстве признаков. За счет этого визуальный энкодер получает более

универсальный сигнал обучения, отражающий не только принадлежность к классу, но и более широкие семантические свойства изображения.

Для задачи, рассматриваемой в данной работе, важно, что после завершения обучения визуальный энкодер CLIP может использоваться отдельно как экстрактор признаков изображений. В этом случае на вход модели подается только изображение, а на выходе получается его эмбединг фиксированной размерности, который далее может использоваться в системе поиска похожих изображений. Формально для изображения x итоговое векторное представление можно записать так:

$$z = \frac{f_{\theta}(x)}{\|f_{\theta}(x)\|}.$$

Далее поиск похожих изображений может выполняться стандартным образом через косинусное сходство или эквивалентное ему на нормированных векторах скалярное произведение. Следовательно, несмотря на мультимодальную природу обучения, CLIP может применяться и в чисто image-to-image постановке как модель генерации эмбедингов изображений.

На рисунке 2.8 представлено краткое изложение подхода к обучению CLIP.

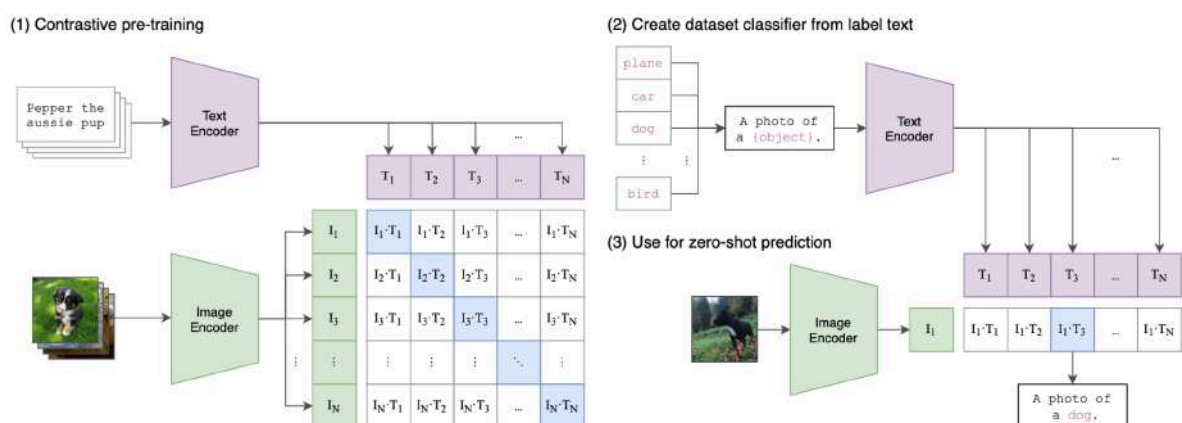


Рисунок 2.8. Визуальное описание обучения модели CLIP

В соответствии с задачами ВКР была реализована и обучена собственная компактная сверточная модель ResNet10TI, предназначенная для дальнейшего использования в качестве extractor-модели в системе визуального поиска. Название модели отражает её связь с residual-подходом и датасетом Tiny ImageNet, использованным для обучения [39]. Основная цель данного этапа состояла не в создании архитектуры, превосходящей крупные предобученные модели, а в подготовке собственной CNN-модели, обученной с нуля и пригодной для извлечения компактных векторных представлений изображений. Такой эксперимент позволяет проверить, что разрабатываемая система работает не только со встроенными моделями, но и с пользовательскими моделями, подготовленными отдельно.

Архитектура ResNet10TI является компактной ResNet-подобной сетью. На вход модели подаётся RGB-изображение размером $3 \times 128 \times 128$. Начальный блок включает свертку 3×3 с 64 выходными каналами, BatchNorm2d и ReLU. Далее последовательно расположены четыре residual-стадии. Первая стадия сохраняет число каналов и пространственное разрешение, формируя выход $64 \times 128 \times 128$. Вторая стадия увеличивает число каналов до 128 и уменьшает пространственное разрешение до 64×64 . Третья стадия формирует тензор $256 \times 32 \times 32$, а четвёртая — $256 \times 16 \times 16$. После этого применяется AdaptiveAvgPool2d с выходным размером 1×1 , затем тензор преобразуется в вектор размерности 256. В классификационном варианте модели этот вектор передаётся в слой Linear(256, 200), поскольку Tiny ImageNet содержит 200 классов. Датасет Tiny ImageNet является более сложным по сравнению с небольшими учебными датасетами, что позволяет обучить модель извлекать разнообразные визуальные признаки. Схематичное устройство модели представлено на рисунке 2.9.

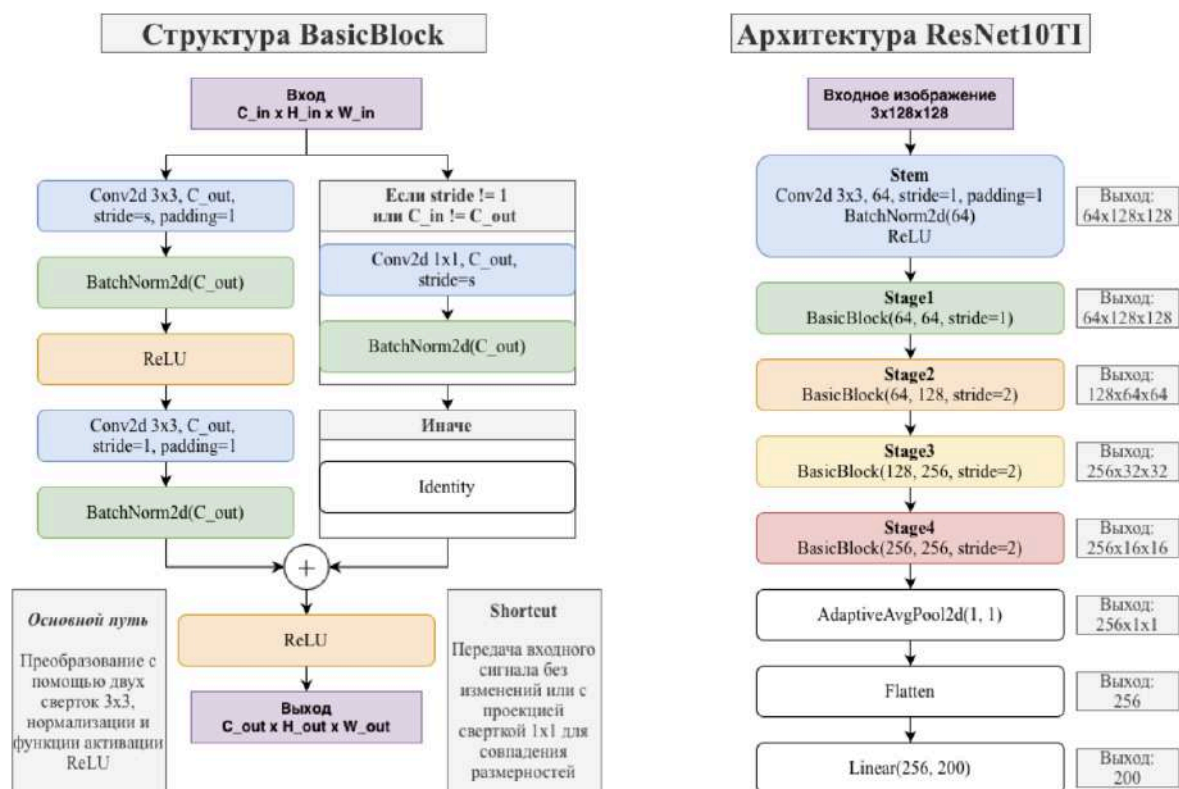


Рисунок 2.9. Архитектура собственной модели ResNet101

Для обучения перед подачей в сеть изображения приводились к размеру 128×128 . Для обучающей выборки использовались Resize до 144×144 , случайный crop 128×128 , случайное горизонтальное отражение и ColorJitter. Для валидационной выборки применялись Resize до 128×128 , преобразование в тензор и нормализация. В обоих случаях использовались средние значения и стандартные отклонения ImageNet.

Обучение проводилось в течение 30 эпох с размером batch, равным 512. В качестве функции потерь использовалась CrossEntropyLoss с label smoothing, равным 0.1. Данный параметр сглаживает целевое распределение классов и снижает чрезмерную уверенность модели в предсказаниях, что положительно сказывается на обобщающей способности. В качестве оптимизатора был выбран AdamW с learning rate $5 \cdot 10^{-4}$ и weight decay 10^{-3} . Для изменения learning rate в процессе обучения использовался планировщик CosineAnnealingLR, постепенно уменьшающий скорость обучения до 10^{-6} . Обучение выполнялось с

использованием GPU, а для ускорения вычислений применялся mixed precision.

В процессе обучения сохранялась лучшая версия модели по значению accuracy на валидационной выборке. Динамика accuracy и функции потерь представлена на рисунке 2.10.

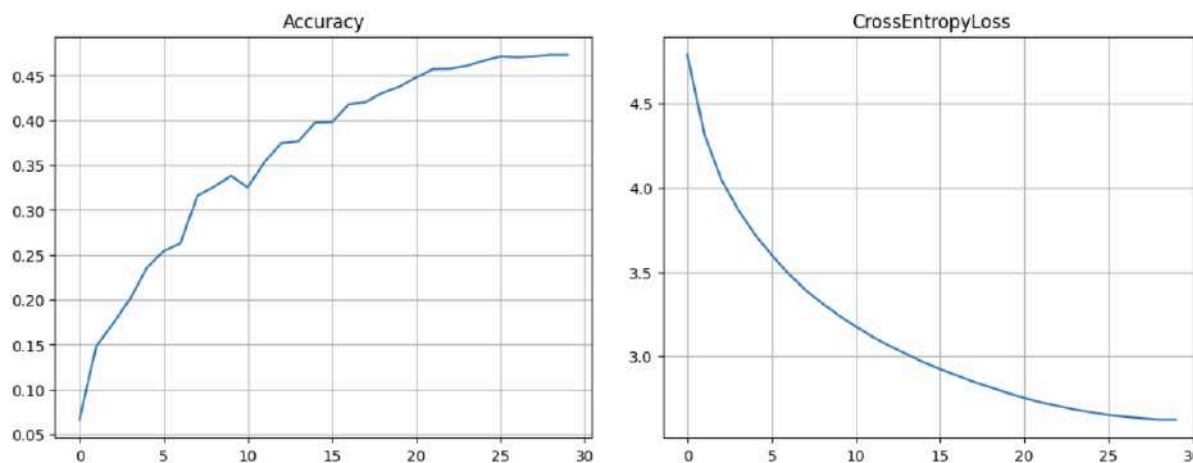


Рисунок 2.10. Кривые обучения модели ResNet101

По кривой CrossEntropyLoss видно, что значение функции потерь последовательно снижалось на протяжении обучения. Кривая accuracy демонстрирует рост качества на валидационной выборке и постепенный выход на плато к последним эпохам. Это указывает на то, что модель усвоила значимую часть визуальных закономерностей Tiny ImageNet, однако её качество остаётся ограниченным из-за компактной архитектуры, обучения с нуля и сложности классификации на 200 классов. К концу обучения точность модели достигла 47,3%, а loss упал до значения 2,3.

После завершения обучения классификационная модель была преобразована в extractor-модель для задачи визуального поиска. Для этого у лучшей сохранённой модели полносвязный классификационный слой fc был заменён на Identity. В результате вместо логитов по 200 классам модель стала возвращать 256-мерный вектор признаков, полученный после глобального усредняющего pooling. Именно этот вектор далее используется как эмбединг изображения в retrieval-системе.

2.2. Требования к системе поиска похожих изображений

Разрабатываемая система предназначена для оценки визуального сходства изображений в локальных коллекциях. В отличие от пользовательских фотоархивов, рассмотренных в главе 1, основная задача приложения состоит не только в отображении похожих изображений, но и в формализованной оценке качества поиска. Поэтому система проектируется как исследовательский инструмент, позволяющий загружать датасет, подключать модель извлечения признаков, рассчитывать эмбединги, выполнять поиск похожих изображений и автоматически вычислять retrieval-метрики.

Базовый режим работы системы строится на поиске по глобальным эмбедингам изображений. Каждое изображение из датасета преобразуется нейросетевой моделью в вектор фиксированной размерности. После этого для выбранного изображения-запроса выполняется сравнение его эмбединга с эмбедингами остальных изображений базы. Результатом является ранжированный список кандидатов, упорядоченный по степени визуального сходства. В рамках данной работы такой подход является основным, поскольку он позволяет напрямую сопоставлять различные модели извлечения признаков и оценивать их качество в единой экспериментальной среде.

Система должна поддерживать загрузку датасета в виде zip-архива. После загрузки архив распаковывается, а приложение определяет структуру содержащихся в нём изображений. Основным поддерживаемым форматом является структура, близкая к ImageFolder, где каждая директория соответствует отдельному классу изображений. Такая организация данных необходима для автоматического определения релевантности: изображения одного класса считаются релевантными друг другу, а изображения разных классов — нерелевантными. Если в архиве не удаётся обнаружить несколько классов, приложение может выполнять

просмотр изображений и поиск похожих объектов, однако расчёт retrieval-метрик в таком режиме становится недоступным.

Система должна поддерживать два варианта работы с моделями извлечения признаков. Первый вариант предполагает загрузку пользовательской модели в формате .pth или .pt. В этом случае модель рассматривается как полный extractor, сохранённый средствами PyTorch, а её метод forward должен возвращать тензор эмбедингов либо структуру, из которой можно извлечь глобальный эмбединг изображения. Вторым вариантом предполагается использование встроенных моделей, загружаемых из открытых репозиториях моделей. Такой режим необходим для проведения сравнительных экспериментов с известными архитектурами без предварительной подготовки собственных моделей.

Для загружаемых пользовательских моделей задаётся стандартный препроцессинг изображения: изменение размера, центральное кадрирование, преобразование в тензор и нормализация по статистикам ImageNet. Для встроенных моделей используется препроцессинг, соответствующий конкретной модели. Это необходимо, поскольку разные архитектуры и режимы предобучения могут предполагать различные параметры нормализации, размер входного изображения и схему обработки данных. Таким образом, препроцессинг является частью экспериментальной конфигурации и должен быть явно зафиксирован при сравнении результатов.

После выбора модели приложение должно рассчитывать эмбединги для всех изображений датасета. Данный этап выполняется батчами, что позволяет использовать GPU-ускорение при наличии CUDA. На выходе формируется матрица эмбедингов размера $N \times D$, где N — число изображений в датасете, а D — размерность векторного представления. Эта матрица используется далее для поиска похожих изображений и расчёта метрик. В рамках данной работы поиск выполняется полным

сравнением изображения-запроса со всеми остальными элементами датасета. Такой подход не требует построения приближённого индекса и позволяет получить точный порядок ранжирования, что особенно важно при проверке корректности метрик на экспериментальных наборах данных ограниченного размера.

Для сравнения эмбедингов в системе предусматривается несколько мер близости: косинусное сходство, скалярное произведение и евклидово расстояние. Выбор меры сравнения не должен быть жёстко зафиксирован в коде, поскольку качество выдачи зависит не только от архитектуры модели, но и от способа сопоставления полученных векторов. Кроме того, приложение должно позволять включать L2-нормализацию эмбедингов перед сравнением. Это важно для корректного сопоставления косинусного сходства, скалярного произведения и евклидова расстояния, поскольку на нормированных векторах эти способы приводят к эквивалентному или близкому порядку ранжирования.

В интерактивном режиме пользователь выбирает одно изображение-запрос и глубину выдачи k . После этого система исключает само изображение-запрос из списка кандидатов и возвращает первые k наиболее похожих изображений. Если датасет имеет классовую структуру, для выбранного запроса дополнительно рассчитываются $\text{Precision}@k$, $\text{Recall}@k$, $\text{Precision}@1$, $\text{HitRate}@k$ и Average Precision . Такой режим позволяет не только визуально оценить качество поиска, но и сразу получить количественную характеристику выдачи для конкретного изображения.

Отдельным требованием является поддержка полного retrieval-обхода датасета. В этом режиме каждое изображение последовательно используется как запрос, а остальные изображения выступают кандидатами для поиска. Для каждого запроса рассчитываются индивидуальные значения метрик, после чего формируется итоговая

таблица усреднённых результатов по всему датасету. В качестве итоговых показателей используются Precision@1, Precision@k, Recall@k, HitRate@k и Mean Average Precision. Такой режим является центральным для экспериментальной части работы, поскольку позволяет сравнивать разные модели и настройки не по отдельным примерам, а по всему набору изображений. Практическая ценность полного обхода состоит также в возможности выявления потенциально проблемных изображений. Если для отдельного изображения-запроса значение AP или Precision@k оказывается низким, такой пример может быть вынесен на дополнительную ручную проверку. Низкое значение метрики не доказывает ошибку разметки автоматически, поскольку причиной может быть сложный ракурс, плохое качество изображения, частичное перекрытие объекта или высокая визуальная близость разных классов. Тем не менее такой подход позволяет сузить область ручного анализа и выделить изображения, наиболее сильно влияющие на качество retrieval-оценки.

Дополнительно в системе должен быть предусмотрен режим reranking для поиска по одному изображению. В отличие от полного обхода датасета, данный режим не используется как основной массовый экспериментальный сценарий, поскольку требует повторного извлечения признаков для изображения-запроса и ограниченного множества кандидатов. Его назначение состоит в интерактивной проверке того, может ли уточнённое сравнение улучшить выдачу для конкретного запроса. В этом режиме сначала выполняется обычный retrieval по глобальным эмбедингам и отбирается top-N кандидатов. Затем выбранная reranking-модель переупорядочивает только это ограниченное множество. Благодаря этому более дорогая процедура сравнения применяется не ко всей базе изображений, а только к наиболее вероятным кандидатам.

Reranking должен выполняться двумя способами. Первый способ использует глобальные эмбединги reranking-модели и повторно

сравнивает изображение-запрос с кандидатами выбранной мерой близости. Второй способ использует patch-level признаки, если модель способна их возвращать. В этом случае изображение представляется не только одним глобальным вектором, но и набором локальных patch-эмбеддингов, а для сравнения изображения-запроса и кандидата будет использоваться симметричная MaxSim схема:

Пусть изображение-запрос q и изображение-кандидат x представлены наборами patch-эмбеддингов:

$$Q = \{q_1, q_2, \dots, q_m\}, X = \{x_1, x_2, \dots, x_n\},$$

где $q_i \in R^d$, $x_j \in R^d$, m и n — число patch-признаков у запроса и кандидата.

Перед сравнением patch-эмбеддинги нормируются по длине:

$$\hat{q}_i = \frac{q_i}{\|q_i\|}, \hat{x}_j = \frac{x_j}{\|x_j\|}.$$

Матрица попарных сходств между patch-признаками запроса и кандидата вычисляется как:

$$S_{ij} = \hat{q}_i^T \hat{x}_j; i = 1, \dots, m; j = 1, \dots, n.$$

Односторонняя MaxSim-оценка от запроса к кандидату (от кандидата к запросу) определяется как среднее максимальное сходство каждого patch-признака запроса (кандидата) с наиболее близким patch-признаком кандидата (запроса):

$$MaxSim(Q \rightarrow X) = \frac{1}{m} \sum_{i=1}^m \max_{1 \leq j \leq n} S_{ij},$$

$$MaxSim(X \rightarrow Q) = \frac{1}{n} \sum_{j=1}^n \max_{1 \leq i \leq m} S_{ij}.$$

Существует удобная устойчивая симметричная MaxSim-оценка, которая усредняет оба направления:

$$MaxSim_{sym}(Q, X) = \frac{1}{2} (MaxSim(Q \rightarrow X) + MaxSim(X \rightarrow Q)).$$

Итоговый score для reranking можно записать как:

$$score(q, x) = MaxSim_{sym}(Q, X).$$

Чем выше значение $score(q, x)$, тем более похожим считается изображение-кандидат x на изображение-запрос q . Такая схема позволяет учитывать локальные соответствия между изображениями и потенциально лучше работать в случаях, когда глобальный эмбединг недостаточно точно отражает различия между близкими объектами.

Важно отметить, что в рамках такой реализации reranking является дополнительным инструментом анализа для одного изображения-запроса, а не частью полного расчёта метрик по всему датасету. Полная экспериментальная оценка выполняется по основному retrieval-этапу, основанному на глобальных эмбедингах. Такое разделение позволяет сохранить вычислительную простоту массового эксперимента и одновременно оставить возможность качественного анализа отдельных запросов с использованием более детального сравнения.

Результаты полного retrieval-обхода должны представляться в двух таблицах, доступных для экспорта в CSV. Первая таблица содержит итоговые метрики эксперимента: название модели, размер входного изображения, размер batch, выбранную меру сходства, режим нормализации и усреднённые значения метрик. Вторая таблица содержит значения метрик для каждого изображения-запроса: путь к изображению, класс, AP, Precision@1, Precision@k, Recall@k и HitRate@k. Такой формат позволяет одновременно анализировать общее качество модели и находить отдельные проблемные примеры.

ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ

В данной главе рассматривается программная реализация разработанной системы оценки сходства изображений для визуального поиска. Описывается архитектура приложения, основные функциональные блоки, порядок загрузки датасета и моделей, расчёт эмбедингов, выполнение поиска похожих изображений, режим полного retrieval-обхода и дополнительный reranking для отдельного запроса. Также внимание уделяется экспериментальной проверке системы: выбору тестового датасета, настройкам экспериментов, сравнению моделей извлечения признаков по retrieval-метрикам и анализу полученных результатов.

Отдельно код и полные результаты экспериментов доступны в созданном репозитории ВКР [38].

3.1. Архитектура приложения для визуального поиска изображений

Программная реализация системы выполнена на языке Python. В качестве интерфейсной основы выбран фреймворк Streamlit, поскольку он позволяет быстро создавать интерактивные веб-приложения вокруг Python-кода без необходимости разрабатывать отдельную клиентскую и серверную части. Streamlit также удобен для запуска в средах наподобие Google Colab, где приложение может быть развернуто непосредственно из ноутбука. Для получения внешней ссылки на приложение используется `pyngrok`. В рамках практической проверки этот способ оказался более стабильным при запуске из Colab по сравнению с `localtunnel` и `Cloudflare Tunnel`, поэтому он был выбран как основной вариант публикации локального Streamlit-сервера.

В начале программы устанавливаются необходимые зависимости `streamlit` и `pyngrok`, после чего выполняется аутентификация `ngrok` с использованием пользовательского `authtoken`. Далее основной код

приложения записывается в файл `app.py`, который затем запускается командой `!streamlit run app.py`. После запуска создаётся публичная ссылка через `ngrok.connect(8501)`, позволяющая открыть интерфейс приложения в браузере. Такая схема делает приложение удобным для демонстрации и тестирования в облачной среде без полноценного развёртывания на отдельном сервере.

В приложении используются библиотеки `torch`, `torchvision`, `PIL`, `pandas`, `streamlit`, а также библиотека `transformers` для загрузки встроенных моделей. `PyTorch` применяется для инференса нейросетевых моделей и операций с эмбедами, `torchvision.transforms` — для препроцессинга изображений, `PIL` — для чтения изображений, `pandas` — для формирования таблиц результатов, а `Streamlit` — для отображения интерфейса и пользовательского взаимодействия. Библиотека `transformers` подключается через блок `try-except`, поэтому приложение может работать и без неё, если используются только пользовательские модели. При этом встроенные модели становятся доступны только при успешном импорте необходимых классов.

Интерфейс приложения поддерживает два языка: русский и английский. Все текстовые элементы вынесены в словарь `TEXTS`, где для каждого сообщения предусмотрены две версии. Такой подход позволяет избежать дублирования логики интерфейса и переключать язык через значение переменной `lang`. После выбора языка приложение отображает название, краткое описание системы и информацию об устройстве, на котором будет выполняться инференс. Устройство определяется автоматически: если доступна `CUDA`, используется `GPU`, иначе вычисления выполняются на `CPU`.

Рабочие файлы приложения размещаются в директории `cbir_workspace`. Внутри неё выделяются подкаталоги для датасета и моделей: `dataset`, `model`, `extracted`. Загруженный `zip`-архив сохраняется как

uploaded_dataset.zip, пользовательская retrieval-модель — как uploaded_retrieval_model.pth, а пользовательская reranking-модель — как uploaded_reranking_model.pth. Такой подход позволяет изолировать временные файлы приложения от остальных файлов рабочей среды и повторно использовать результаты между перезапусками Streamlit внутри одной сессии.

Для батчевой обработки изображений реализован класс ImagePathDataset, наследующийся от torch.utils.data.Dataset. Он принимает список путей к изображениям и функцию преобразования изображения в тензор. Метод `__len__` возвращает количество изображений, а метод `__getitem__` открывает изображение по индексу, переводит его в RGB-формат, применяет препроцессинг и возвращает пару из тензора изображения и строкового пути к файлу. Этот класс используется при расчёте эмбедингов по всему датасету и при извлечении признаков для reranking.

Для работы со встроенными моделями реализованы три класса-обёртки. Класс RecommendedViTExtractor используется для моделей ViT и DINO на базе ViTModel. В методе forward модель возвращает словарь с двумя элементами: global, соответствующий CLS-токену, и patches, соответствующий patch-токенам изображения. Класс RecommendedDINOv2Extractor аналогично оборачивает модели DINOv2, загружаемые через AutoModel. При извлечении patch-признаков он учитывает возможное наличие register-токенов и исключает их из набора patch-эмбедингов. Класс RecommendedCLIPVisionExtractor предназначен для vision-части CLIP. Он возвращает глобальный эмбединг изображения image_embeds и patch-признаки, дополнительно обработанные через post_layernorm и visual_projection, чтобы привести их к пространству признаков CLIP.

Список встроенных моделей задаётся словарём `RECOMMENDED_MODEL_CONFIGS`. В него включены supervised ViT-модели, DINO, DINOv2 и CLIP-модели. Для каждой модели указаны тип и идентификатор в репозитории Hugging Face. Это позволяет пользователю выбрать модель из интерфейса, после чего приложение автоматически загружает соответствующий backbone и препроцессинг. Функция `build_recommended_model` по выбранной конфигурации создаёт нужную обёртку, а функция `load_recommended_model_cached` загружает модель на устройство, переводит её в режим eval и кэширует через `st.cache_resource`. Кэширование необходимо, поскольку Streamlit перезапускает скрипт при каждом изменении состояния интерфейса, и без кэша модель загружалась бы повторно.

Для пользовательских моделей реализована функция `load_uploaded_model`. Она сохраняет загруженный файл в рабочую директорию, загружает модель через `torch.load`, переносит её на выбранное устройство и переводит в режим инференса. В текущей реализации предполагается, что пользовательская модель сохранена как полный объект модели, а не только как `state_dict`. Это ограничение упрощает загрузку, поскольку приложению не требуется заранее знать архитектуру пользовательской модели.

Отдельный блок функций отвечает за работу с файловой системой и структурой датасета. Функция `clear_directory` полностью очищает указанную директорию и создаёт её заново. Функция `extract_zip` распаковывает zip-архив. Функция `build_uploaded_file_state_key` вычисляет ключ состояния загруженного файла на основе имени, размера и SHA-256 хэша содержимого. Этот ключ используется для определения того, изменился ли загруженный архив или модель. Если файл не изменился, приложение не выполняет повторную распаковку и не сбрасывает состояние без необходимости.

Для фильтрации файлов и директорий используются функции `is_hidden_or_system_name`, `is_valid_dir` и `is_valid_image_file`. Они исключают скрытые системные файлы, служебные директории и неподдерживаемые расширения. Поддерживаемыми форматами изображений являются `.jpg`, `.jpeg`, `.png`, `.bmp` и `.webp`. Функция `count_images_recursively_in_directory` подсчитывает изображения внутри директории, а `collect_all_images_recursively` собирает все изображения из указанной директории рекурсивно.

Определение структуры датасета выполняет функция `detect_dataset_structure`. Она анализирует содержимое распакованного архива и поддерживает два основных варианта классовой структуры. Первый вариант соответствует схеме `extract_dir / dataset_name / class_1 / class_2`, когда внутри архива находится одна общая папка датасета, а уже внутри неё расположены директории классов. Второй вариант соответствует схеме `extract_dir / class_1 / class_2`, когда директории классов находятся непосредственно в корне распаковки. В обоих случаях `class-based` режим активируется только тогда, когда найдено больше одной директории с изображениями. Если обнаружен только один класс или классы не определены, приложение переходит в режим общего списка изображений без расчёта метрик.

После определения структуры датасета формируется индекс изображений. Если классы найдены, используется функция `build_class_dataset_index`, возвращающая словарь вида «имя класса — список путей к изображениям». Если классовая структура не обнаружена, применяется функция `build_global_image_index`, возвращающая единый список изображений в группе `all_images`. Далее функция `flatten_dataset_index` преобразует индекс в общий список путей, а `build_path_to_label` строит отображение от пути изображения к его классу.

Это отображение используется при расчёте релевантности в retrieval-метриках.

Для безопасного отображения изображений реализована функция `load_image_safely`. Она пытается открыть изображение через PIL и возвращает объект изображения, а при ошибке возвращает `None`. Это позволяет приложению не завершаться аварийно при наличии повреждённых файлов. Функция `relative_path_for_display` формирует относительный путь изображения для отображения в интерфейсе, а `structure_mode_to_text` возвращает человекочитаемое описание найденной структуры датасета.

Препроцессинг изображений реализован двумя способами. Для пользовательских моделей используется функция `build_preprocess`, которая задаёт стандартную схему: `Resize`, `CenterCrop`, преобразование в тензор и нормализация по статистикам ImageNet. Размер входного изображения выбирается пользователем в интерфейсе. Для встроенных моделей используется функция `build_recommended_preprocess_cached`, которая загружает `AutoImageProcessor` выбранной модели и извлекает из него параметры размера, кадрирования, среднего и стандартного отклонения. Вспомогательная функция `get_processor_size_value` нужна для корректной обработки разных форматов параметров `size` и `crop_size`.

Для работы с выходами моделей реализован набор функций нормализации и извлечения тензоров. Функция `get_first_tensor_from_output` позволяет извлечь первый тензор из разных типов выхода: обычного тензора, словаря, списка, кортежа или объекта с полями `image_embeds`, `pooler_output`, `last_hidden_state`. Функция `get_global_tensor_from_output` сначала пытается найти ключ `global`, а при его отсутствии использует общий fallback-механизм. Функция `get_patch_tensor_from_output` извлекает patch-признаки, если они явно присутствуют в словаре по ключу `patches` или доступны через `last_hidden_state`. Функции

`normalize_global_embedding_shape` и `normalize_patch_embedding_shape` приводят глобальные и patch-эмбеддинги к ожидаемой форме: $[B, D]$ для глобальных признаков и $[B, N, D]$ для patch-признаков.

Дополнительно реализована функция `infer_model_tensor_shapes`, которая выполняет пробный forward-pass на искусственном изображении и определяет форму входного тензора и выходных тензоров модели. Эта информация отображается в интерфейсе для reranking-модели. Такой механизм полезен для пользователя, поскольку позволяет понять, возвращает ли модель только глобальный эмбеддинг или также patch-признаки.

Основной расчёт эмбеддингов выполняет функция `compute_dataset_embeddings`. Она получает модель, список изображений, препроцессинг, размер batch и устройство. Внутри создаётся объект `ImagePathDataset` и `DataLoader`, после чего изображения батчами пропускаются через модель в режиме `torch.no_grad()`. Из выхода модели извлекается глобальный эмбеддинг, приводится к форме $[B, D]$, переносится на CPU и добавляется в общий список. После обработки всех батчей эмбеддинги объединяются в матрицу `embeddings_matrix`, а также возвращается список путей `embedding_paths`. Эти два объекта составляют основу дальнейшего поиска.

Для reranking используется функция `extract_features_for_paths`. В отличие от `compute_dataset_embeddings`, она извлекает не только глобальные признаки, но и patch-признаки, если модель их возвращает. Эта функция применяется не ко всему датасету, а только к изображению-запросу и ограниченному числу retrieval-кандидатов. Такой подход снижает вычислительную стоимость reranking, поскольку более детальное сравнение выполняется только для top-N изображений.

Подготовка эмбеддингов к сравнению выполняется функцией `prepare_embeddings_for_search`. Если выбран метод `cosim`, эмбеддинги

обязательно L2-нормализуются. Для dot product и euclidean distance нормализация может включаться или отключаться пользователем. Функция compute_similarity_scores рассчитывает значения сходства или расстояния между изображением-запросом и всеми изображениями датасета. Для косинусного сходства и скалярного произведения используется матричное умножение, а для евклидова расстояния — torch.cdist. Функция также возвращает признак того, нужно ли сортировать значения по убыванию или по возрастанию.

Ранжирование кандидатов выполняет функция rank_all_candidate_images. Она получает score для всех изображений, исключает само изображение-запрос и сортирует оставшиеся кандидаты. Для мер сходства лучшие кандидаты имеют большие значения, а для евклидова расстояния — меньшие. Функция find_similar_images является более узкой обёрткой над полным ранжированием и возвращает только первые top_k кандидатов. В текущей реализации поиск выполняется полным сравнением по всей матрице эмбедингов.

Режим reranking реализован функциями compute_global_rerank_scores, compute_patch_maxsim_scores и rerank_candidate_indices. В режиме global reranking изображение-запрос и кандидаты повторно сравниваются по глобальным эмбедингам reranking-модели. В режиме patch reranking используется симметричная MaxSim-оценка. Функция rerank_candidate_indices объединяет весь процесс: выбирает пути top-N кандидатов, извлекает признаки, рассчитывает новые score и возвращает переупорядоченные индексы.

Расчёт retrieval-метрик реализован отдельным блоком функций. Функция compute_metrics_from_relevance принимает список бинарной релевантности и рассчитывает Precision@k, Recall@k, Precision@1, HitRate@k и Average Precision. Если для изображения-запроса отсутствуют другие изображения того же класса, метрики возвращаются как

недоступные. Функция `compute_metrics_from_ranked_indices` строит список релевантности по готовому ранжированному списку индексов, сравнивая метку каждого кандидата с меткой запроса. Функция `compute_query_metrics` выполняет полный retrieval-обход для одного изображения и рассчитывает метрики по полученному ранжированию.

Полный обход датасета выполняет функция `evaluate_full_dataset`. Она последовательно использует каждое изображение как запрос, рассчитывает для него retrieval-метрики и формирует таблицу per-query результатов. Затем из валидных запросов вычисляется итоговая таблица усреднённых метрик: $P@1$, $P@k$, $R@k$, $\text{HitRate}@k$ и mAP . В итоговую таблицу также добавляются параметры эксперимента. Первая таблица предназначена для сравнения моделей и настроек, а вторая — для анализа отдельных изображений, включая потенциально проблемные элементы датасета. Для экспорта результатов используется функция `dataframe_to_csv_bytes`, сохраняющая CSV в кодировке UTF-8 with BOM, что упрощает открытие таблиц в Excel.

Состояние приложения управляется через `st.session_state`. В нём хранятся ключ загруженного датасета, результат расчёта эмбеддингов, ключ состояния эмбеддингов, результат поиска по одному изображению, результат полного обхода датасета и тип активного результата. Такая организация необходима из-за особенностей Streamlit: при каждом действии пользователя скрипт выполняется заново. Использование `session_state` позволяет не терять уже рассчитанные эмбеддинги и результаты экспериментов, если параметры не изменились. При загрузке нового датасета или выборе другой модели старые эмбеддинги и результаты сбрасываются, что предотвращает использование неактуальных данных.

Пользовательский сценарий приложения начинается с загрузки zip-архива. Если архив отсутствует, приложение выводит информационное

сообщение и останавливается. После загрузки архив сохраняется и распаковывается, а при некорректном zip-файле выводится ошибка. Далее приложение определяет структуру датасета, строит индекс изображений и отображает общую статистику: количество классов и количество изображений. В отдельном раскрывающемся блоке показывается техническая информация о структуре датасета и корневом пути. Если классы обнаружены, приложение отображает распределение изображений по классам и позволяет просматривать изображения выбранного класса. Если классы не обнаружены, включается режим общего просмотра изображений без подсчёта метрик. Пример начала работы с приложением показан в приложении 1 (рисунки 1-3).

После просмотра данных пользователь выбирает retrieval-модель. При выборе пользовательской модели отображается загрузчик .pth или .pt и пояснение к ожидаемому формату модели. При выборе встроенной модели пользователь выбирает одну из доступных архитектур. Затем задаются параметры препроцессинга и инференса. После успешной загрузки модели пользователь нажимает кнопку расчёта эмбедингов. Приложение измеряет общее время обработки, скорость в изображениях в секунду и среднее время на одно изображение, после чего отображает размерность полученных эмбедингов. Пример загрузки пользовательской модели и подсчёта эмбедингов представлен в приложении 1 (рисунки 4-6).

Далее пользователь выбирает изображение-запрос из текущего класса или общего списка изображений. Для поиска задаются мера сравнения, режим L2-нормализации и глубина выдачи top-k. После этого можно выполнить либо поиск похожих изображений для одного запроса, либо полный retrieval-обход датасета. Поиск по одному изображению возвращает top-k выдачу, отображает найденные изображения, значения score и, при наличии классовой структуры, retrieval-метрики для выбранного запроса. Полный обход датасета формирует две таблицы:

итоговые метрики эксперимента и метрики по каждому изображению-запросу. Обе таблицы можно скачать в формате CSV. Примеры выбора изображения-запроса, задания параметров работы системы, а также выдачи похожих изображений для одного запроса и метрик для всего датасета находятся в приложении 1 (рисунки 7-9).

Режим `reranking` настраивается отдельно и применяется только к поиску по одному изображению. Пользователь может отключить `reranking`, загрузить собственную `reranking`-модель или выбрать встроенную модель. Затем выбирается тип признаков: глобальные эмбединги или `patch`-признаки. Также задаётся число `retrieval`-кандидатов `top-N`, которые будут переданы на переупорядочивание. После запуска поиска приложение сначала получает полное `retrieval`-ранжирование по основной модели, затем переупорядочивает первые `top-N` кандидатов `reranking`-моделью и отображает итоговую выдачу. Если выбран `patch-level` режим, но модель не возвращает `patch`-признаки, приложение выводит соответствующее сообщение об ошибке. Примеры работы приложения с `reranking`-шагом представлены в приложении 1 (рисунки 10-13).

Такая структура приложения соответствует исследовательской постановке работы, поскольку позволяет не только визуально искать похожие изображения, но и количественно сравнивать модели извлечения признаков, анализировать влияние параметров поиска и выявлять изображения с низким качеством `retrieval`-выдачи.

3.2. Экспериментальное исследование качества поиска и оценка работы системы

Для экспериментальной проверки разработанной системы использовался датасет `cat-breeds`, размещённый на платформе `Hugging Face` [37]. Датасет содержит 4000 изображений и 4 класса пород кошек, в каждом классе находится по 1000 изображений. Выбор именно этого

датасета обусловлен несколькими причинами. Во-первых, задача поиска изображений одной породы кошек относится к *fine-grained retrieval*: все изображения принадлежат к одной общей семантической категории «кошка», однако классы различаются породой. Следовательно, модели недостаточно выделить общий объект на изображении; она должна сохранять в эмбединге более тонкие визуальные признаки, связанные с формой морды, окрасом, шерстью и другими особенностями породы. Во-вторых, датасет является сбалансированным, поэтому итоговые метрики не искажаются сильным различием в числе изображений между классами. В-третьих, объём в 4000 изображений достаточен для полноценного полного *retrieval*-обхода, но при этом остаётся приемлемым для запуска в среде Google Colab без построения приближённого индекса ближайших соседей. В-четвёртых, изображения кошек хорошо подходят для визуальной интерпретации результатов: найденные системой ошибки можно проверить не только по таблицам метрик, но и по самим изображениям.

Для всех экспериментов использовался единый тестовый сценарий. Основной глубиной выдачи было выбрано значение $k = 10$, поскольку оно позволяет оценить качество верхней части ранжированного списка и одновременно остаётся удобным для визуального анализа результатов. Для большинства встроенных моделей применялось центральное кадрирование до размера 224×224 пикселей, так как этот размер является распространённым стандартом для ResNet, ViT и ряда других моделей компьютерного зрения. При этом для пользовательских моделей размер входа задавался отдельно: собственная модель ResNet10TI обучалась и использовалась с входным разрешением 128×128 пикселей. Размер batch по умолчанию был установлен равным 32. Этот параметр не влияет на математическое определение *retrieval*-метрик, однако batch size влияет на скорость расчёта эмбедингов и потребление памяти GPU. Значение 32

было выбрано как компромисс между эффективным использованием GPU и устойчивостью запуска в среде Colab. Меньшие значения batch приводят к большему числу итераций и увеличению накладных расходов, тогда как слишком большой batch может привести к нехватке видеопамяти, особенно при использовании трансформерных моделей. Поэтому batch size 32 использовался как базовая настройка при сравнении моделей.

Экспериментальная часть была разделена на три блока. Первый блок посвящён сравнению скорости инференса моделей. Второй блок — влиянию выбранного метода сравнения эмбеддингов и L2-нормализации. Третий блок представляет основное сравнение качества retrieval между разными моделями.

Первый эксперимент был направлен на оценку скорости работы моделей при расчёте эмбеддингов: на практике важно учитывать, насколько быстро модель способна обработать изображение. Результаты данного эксперимента следует интерпретировать совместно с метриками качества, так как более быстрая модель может быть предпочтительней, если её качество лишь незначительно уступает более тяжёлой модели, и напротив, если более крупная модель даёт существенный прирост mAP и Precision@10, увеличение времени инференса может быть оправдано. Поэтому в работе скорость инференса рассматривается не как самостоятельная итоговая метрика качества поиска, а как дополнительная характеристика применимости модели в интерактивной системе. В таблице 1 представлены итоги этого эксперимента.

Таблица 1. Результаты измерения скорости инференса моделей

Модель	Размер эмбеддинга	Время на весь датасет, сек	Изображений /сек	Секунд/ изображение
resnet10ti	256	21,2844	187,9306	0,005321

resnet18	512	22,7465	175,851	0,005687
vitb_patch32	768	28,956	138,1405	0,007239
resnet50	2048	29,1257	137,3356	0,007281
dino_vits16	384	30,0866	132,9493	0,007522
clipvb_patch32	512	30,4485	131,3695	0,007612
dinov2s	384	37,3467	107,1045	0,009337
vitb_patch16	768	64,3576	62,1527	0,016089
dino_vitb16	768	65,5906	60,9844	0,016398
clipvb_patch16	512	66,2365	60,3897	0,016559
dinov2b	768	91,0576	43,9283	0,022764

По итогам эксперимента самой быстрой моделью оказалась ResNet101T: она обработала датасет за 21,2844 секунды со скоростью 187,9306 изображения в секунду. Такой результат связан с компактной архитектурой модели и меньшей размерностью итогового эмбединга — 256. ResNet-18 оказалась близка по скорости и обработала датасет за 22,7465 секунды, формируя при этом 512-мерные эмбединги. ResNet-50 потребовала 29,1257 секунды, но её выходное представление имеет размерность 2048, что увеличивает объём вычислений и хранения признаков. Среди transformer-моделей быстрее всего работали варианты с patch size 32: ViT-B/32 обработала датасет за 28,9560 секунды, а CLIP ViT-B/32 — за 30,4485 секунды. Это объясняется меньшим числом patch-токенов по сравнению с моделями patch size 16. При размере входа 224×224 модель с patch size 16 получает 196 patch-токенов, тогда как модель с patch size 32 — только 49 patch-токенов. DINOv2-S/14 также показала приемлемый результат: 37,3467 секунды на датасет. Наиболее

медленной моделью оказалась DINOv2-B/14: время обработки составило 91,0576 секунды, а скорость — 43,9283 изображения в секунду. Такая вычислительная стоимость связана с более крупной архитектурой и меньшим размером patch, что увеличивает число обрабатываемых токенов.

Следует отметить ограничения данного эксперимента. Замер времени выполнялся один раз для каждой модели, поэтому результаты не являются строгим сравнением. На время инференса могут влиять тип и состояние GPU, фоновая нагрузка среды выполнения, скорость чтения изображений, кэширование данных и версии библиотек. Кроме того, для разных моделей могут отличаться входное разрешение, препроцессинг и размер выходного эмбединга, что также влияет на итоговое время обработки. Для более строгой оценки требовалось бы провести несколько повторов, исключить прогревочный запуск и приводить среднее значение времени со стандартным отклонением. В рамках данной работы эксперимент используется как практическая ориентировочная оценка применимости моделей в интерактивном приложении, а не как независимое исследование их производительности.

Второй эксперимент был посвящён сравнению методов сопоставления эмбедингов. Он проводился на модели ResNet-18, для которой были проверены косинусное сходство, скалярное произведение и евклидово расстояние в режимах с L2-нормализацией и без неё. Цель эксперимента состояла не в выборе лучшей модели, а в проверке теоретического свойства, рассмотренного в главе 1.2: на L2-нормированных векторах косинусное сходство, скалярное произведение и евклидово расстояние должны давать одинаковый или практически одинаковый порядок ранжирования. Полученные результаты, представленные в таблице 2, подтвердили это свойство.

Таблица 2. Эксперимент на одинаковый результат при L2-нормализации

similarity	normalize	P@1	P@10	R@10	HitRate@10	mAP
cosine	True	0,75	0,71092	0,007116	0,974	0,50432
dot	True	0,75	0,71092	0,007116	0,974	0,50432
euclidean	True	0,75	0,71092	0,007116	0,974	0,50432
dot	False	0,56825	0,4396	0,0044	0,9973	0,38540
euclidean	False	0,74025	0,698	0,006987	0,9763	0,46577

Третий эксперимент являлся основным и был направлен на сравнение качества retrieval у разных моделей извлечения признаков. В отличие от второго эксперимента, где влияние методов сравнения анализировалось только на примере ResNet-18, в данном блоке оценивались все рассматриваемые модели и следующие варианты сопоставления эмбеддингов: косинусное сходство с L2-нормализацией, скалярное произведение без нормализации и евклидово расстояние без нормализации. Такой подход позволил оценить не только качество самих признаков пространств, но и устойчивость моделей к выбору меры близости. Результаты эксперимента представлены в приложении 2 (таблица 1).

Наилучший результат в основном эксперименте показала модель DINOv2-B/14. Максимальное значение mAP было получено при использовании евклидова расстояния без нормализации и составило 0,7015. При косинусном сходстве с L2-нормализацией та же модель получила mAP = 0,6991, а при скалярном произведении без нормализации — 0,6984. Различия между этими значениями невелики, что указывает на устойчивость признакового пространства DINOv2-B/14 к выбору меры сравнения.

Второе место заняла модель DINOv2-S/14. Для неё значения mAP также оказались близкими при разных способах сравнения: 0,6590 для косинусного сходства, 0,6587 для евклидова расстояния и 0,6580 для скалярного произведения. Это показывает, что даже меньшая версия DINOv2 формирует достаточно стабильные эмбединги, для которых итоговое качество retrieval слабо зависит от конкретной меры близости. При этом DINOv2-S/14 заметно уступает DINOv2-B/14 по mAP, но сохраняет высокие значения Precision@1 и Precision@10.

Модели DINO первого поколения показали более низкие значения mAP по сравнению с DINOv2, однако также заняли верхнюю часть таблицы. Для DINO ViT-B/16 лучший результат был получен при евклидовом расстоянии без нормализации: mAP = 0,5860. Косинусное сходство дало близкий результат mAP = 0,5837, а скалярное произведение без нормализации — 0,5798. Для DINO ViT-S/16 наблюдается аналогичная картина: mAP = 0,5765 при евклидовом расстоянии, 0,5752 при косинусном сходстве и 0,5727 при скалярном произведении. Следовательно, DINO-модели также обладают достаточно устойчивым признаковым пространством, но уступают DINOv2 по качеству ранжирования релевантных изображений.

Сверточная модель ResNet-50 показала лучший результат при косинусном сходстве с L2-нормализацией: mAP = 0,5724. При евклидовом расстоянии без нормализации качество снизилось до mAP = 0,5274, а при скалярном произведении без нормализации — до mAP = 0,4917. Это показывает, что для ResNet-50 нормализация эмбедингов существенно важнее, чем для моделей DINO и DINOv2. Без нормализации длина вектора начинает сильнее влиять на порядок ранжирования, из-за чего качество поиска падает. Аналогичная картина наблюдается для ResNet-18: при косинусном сходстве с нормализацией mAP составил 0,5043, при

евклидовом расстоянии без нормализации — 0,4658, а при скалярном произведении без нормализации — только 0,3854.

Отдельно была оценена собственная сверточная модель ResNet10TI, обученная с нуля на Tiny ImageNet и использованная в приложении как пользовательская extractor-модель. Лучший результат для неё был получен при косинусном сходстве с L2-нормализацией: $mAP = 0,3567$, $Precision@1 = 0,5890$ и $Precision@10 = 0,5293$. При евклидовом расстоянии без нормализации качество оказалось близким и составило $mAP = 0,3526$, тогда как при скалярном произведении без нормализации значение mAP снизилось до 0,2885. Это показывает, что ResNet10TI, как и другие сверточные модели, чувствительна к выбору меры сравнения и особенно к отсутствию нормализации при использовании dot product. При этом более низкое качество по сравнению с предобученными моделями ожидаемо, поскольку ResNet10TI имеет компактную архитектуру, 256-мерный эмбединг и обучалась с нуля на ограниченном по масштабу датасете. Её значение в эксперименте состоит в демонстрации полного сценария подключения собственной модели к разработанной системе визуального поиска.

Результаты обычных ViT-моделей оказались менее однозначными. Для ViT-B/16 максимальное значение mAP было получено при скалярном произведении без нормализации и составило 0,5595, тогда как косинусное сходство с нормализацией дало $mAP = 0,5420$, а евклидово расстояние — 0,5102. При этом $Precision@1$ у ViT-B/16 выше именно при косинусном сходстве: 0,7745 против 0,7510 при скалярном произведении. Это означает, что скалярное произведение без нормализации лучше распределило релевантные изображения по ранжированному списку в среднем по AP, но хуже сработало для самого первого результата. Для ViT-B/32 лучший mAP также получен при скалярном произведении без нормализации — 0,5384,

тогда как косинусное сходство дало 0,5222, а евклидово расстояние — 0,4933.

Модели CLIP показали наиболее низкие значения mAP среди сравниваемых встроенных моделей. Для CLIP ViT-B/16 лучший результат был получен при евклидовом расстоянии без нормализации: mAP = 0,4169. Косинусное сходство дало близкое значение mAP = 0,4143, а скалярное произведение без нормализации заметно уступило — mAP = 0,3711. Для CLIP ViT-B/32 наблюдается аналогичная картина: mAP = 0,4138 при евклидовом расстоянии, 0,4121 при косинусном сходстве и 0,3705 при скалярном произведении. При этом HitRate@10 у CLIP остаётся высоким: от 0,9798 до 0,9893. Это означает, что модель часто помещает хотя бы одно релевантное изображение в первую десятку, но хуже упорядочивает все релевантные изображения внутри выдачи. Такой результат можно объяснить природой обучения CLIP: модель формирует универсальное мультимодальное пространство признаков, хорошо отражающее общий семантический смысл изображения, но менее точно подходящее для fine-grained поиска по изображению между породами кошек.

По итогам основного эксперимента можно сделать несколько выводов. Во-первых, лучшие результаты показали модели семейства DINOv2, причём DINOv2-B/14 оказалась лидером по mAP, Precision@1 и Precision@10. Во-вторых, DINO и DINOv2 продемонстрировали высокую устойчивость к выбору меры сходства: значения метрик при cosine, euclidean и dot различались незначительно. В-третьих, ResNet-модели оказались значительно чувствительнее к нормализации: при скалярном произведении без L2-нормализации качество заметно снижалось. В-четвёртых, собственная модель ResNet10TI уступила крупным предобученным моделям по качеству ранжирования, но подтвердила возможность использования в системе самостоятельно обученного extractor и показала ожидаемый компромисс между компактностью, скоростью и

качеством. В-пятых, обычные ViT-модели показали промежуточные результаты, причём для них скалярное произведение без нормализации дало более высокий mAP, но не всегда лучший Precision@1. В-шестых, CLIP-модели показали высокий HitRate@10, но низкий mAP, что говорит о способности находить хотя бы один релевантный объект в верхней части выдачи, но о недостаточно точном полном ранжировании релевантных изображений.

Отдельно следует отметить низкие абсолютные значения Recall@10. Поскольку в каждом классе содержится 1000 изображений, для каждого изображения-запроса существует 999 релевантных кандидатов. При глубине выдачи $k=10$ максимальное возможное значение Recall@10 составляет приблизительно $10/999$, то есть около 0.01. Поэтому значения Recall@10 на уровне 0.006–0.008 не означают низкое качество поиска, а отражают ограниченную глубину выдачи относительно общего числа релевантных изображений. В данной постановке более информативными метриками являются Precision@1, Precision@10 и mAP.

В целом основной эксперимент подтвердил, что разработанное приложение позволяет сравнивать не только разные модели извлечения признаков, но и разные способы сопоставления эмбеддингов. На датасете cat_breeds_4 наилучшее качество продемонстрировали self-supervised модели DINOv2, что указывает на высокую пригодность таких представлений для поиска визуально похожих изображений в условиях fine-grained классов. При этом результаты также показывают, что выбор меры сходства и нормализации может заметно влиять на качество отдельных моделей, поэтому эти параметры необходимо фиксировать в экспериментальной конфигурации.

ЗАКЛЮЧЕНИЕ

В рамках работы разработана локальная система оценки сходства изображений для задач визуального поиска, ориентированная на исследовательские сценарии. Приложение позволяет загружать датасеты в структуре ImageFolder, подключать как пользовательские модели в форматах .pth или .pt, так и встроенные модели из семейств ViT, DINO, DINOv2 и CLIP, рассчитывать эмбединги, выполнять поиск похожих изображений и автоматически вычислять ключевые метрики ранжирования: Precision@k, Recall@k, Precision@1, HitRate@k, Average Precision и Mean Average Precision. Реализованы полный retrieval-обход датасета с экспортом сводных и поэкземплярных таблиц в CSV, а также режим reranking для одного запроса по глобальным или patch-признакам.

Экспериментальная проверка на сбалансированном fine-grained датасете пород кошек выявила, что наилучшее качество поиска обеспечивает модель DINOv2-B/14 ($mAP = 0,7015$), демонстрирующая высокую устойчивость к выбору меры сравнения. Модель DINOv2-S/14 показала близкие результаты при лучшем соотношении качества и скорости. Сверточная ResNet-50 достигла $mAP = 0,5724$ при косинусном сходстве с нормализацией, однако её качество существенно падает без L2-нормализации эмбедингов, в отличие от DINO и DINOv2. Собственная компактная модель ResNet101T, обученная с нуля на Tiny ImageNet, подтвердила возможность интеграции пользовательских экстракторов, но ожидаемо уступила крупным предобученным архитектурам, достигнув значения $mAP 0,3567$. CLIP-модели показали сравнительно низкий mAP при высоком HitRate — они часто находят хотя бы один релевантный результат, но хуже ранжируют весь список в условиях тонких межклассовых различий. Эксперимент с мерами сходства подтвердил, что при L2-нормализации косинусное сходство, скалярное произведение и евклидово расстояние дают одинаковый порядок ранжирования.

Практическая значимость системы заключается в возможности использовать её как локальный инструмент для сравнительного анализа моделей визуального поиска, подбора оптимальных параметров (мера сходства, нормализация), диагностики разметки датасетов через выявление изображений с аномально низкими метриками, а также для воспроизводимой оценки качества retrieval-пайплайна. В дальнейшем система может быть расширена за счёт добавления FAISS или другого ANN-индекса для масштабируемого поиска, поддержки новых self-supervised и foundation-моделей, количественной оценки reranking на всём датасете, сохранения истории экспериментов, интеграции с векторными базами данных и возможности загрузить отдельную картинку как изображение-запрос без включения её в структуру датасета.

Данная работа выполнена мною самостоятельно.

«__» _____ 20__ г.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Baranchuk D. Text-to-Image dataset for billion-scale similarity search / D. Baranchuk, A. Babenko // Yandex Research. — 2021. — URL: <https://research.yandex.com/datasets/text-to-image-dataset-for-billion-scale-similarity-search> (дата обращения: 12.02.2026). — Текст: электронный.
2. Bing Image Search documentation // Microsoft Docs: GitHub repository. — URL: <https://github.com/MicrosoftDocs/bing-docs/tree/main/bing-docs/bing-image-search> (дата обращения: 17.02.2026). — Текст: электронный.
3. Bing Visual Search documentation // Microsoft Docs: GitHub repository. — URL: <https://github.com/MicrosoftDocs/bing-docs/tree/main/bing-docs/bing-visual-search> (дата обращения: 17.02.2026). — Текст: электронный.
4. Duplicate detection // Immich Documentation. — URL: <https://docs.immich.app/administration/system-settings/#duplicate-detection> (дата обращения: 18.02.2026). — Текст: электронный.
5. Duplicates // PhotoPrism Documentation. — URL: <https://docs.photoprism.app/user-guide/library/duplicates/> (дата обращения: 19.02.2026). — Текст: электронный.
6. Face recognition // PhotoPrism Documentation. — URL: <https://docs.photoprism.app/developer-guide/vision/face-recognition/> (дата обращения: 19.02.2026). — Текст: электронный.
7. Jobs and workers // Immich Documentation. — URL: <https://docs.immich.app/administration/jobs-workers/> (дата обращения: 18.02.2026). — Текст: электронный.
8. Maintenance duplicates // digiKam Documentation. — URL: https://docs.digikam.org/en/maintenance_tools/maintenance_duplicates.html (дата обращения: 20.02.2026). — Текст: электронный.

9. Object Detection in Visual Search // Bing Search Quality Insights. — 2017.
— URL:
<https://blogs.bing.com/search-quality-insights/2017-09/Object-Detection-in-Visual-Search> (дата обращения: 17.02.2026). — Текст: электронный.
10. Originals // PhotoPrism Documentation. — URL:
<https://docs.photoprism.app/user-guide/library/originals/> (дата обращения: 19.02.2026). — Текст: электронный.
11. Perceptual hashes // PhotoPrism Documentation. — URL:
<https://docs.photoprism.app/developer-guide/metadata/perceptual-hashes/>
(дата обращения: 19.02.2026). — Текст: электронный.
12. Prokhorenkova L. Graph-based nearest neighbor search / L. Prokhorenkova, D. Baranchuk // Yandex Research. — 2022. — URL:
<https://research.yandex.com/blog/graph-based-nearest-neighbor-search> (дата обращения: 13.02.2026). — Текст: электронный.
13. Query parameters // Microsoft Docs : GitHub repository. — URL:
<https://github.com/MicrosoftDocs/bing-docs/blob/main/bing-docs/bing-image-search/reference/query-parameters.md> (дата обращения: 17.02.2026). — Текст: электронный.
14. SafeSearch and your website // Google Search Help. — URL:
<https://support.google.com/websearch/answer/510> (дата обращения: 16.02.2026). — Текст: электронный.
15. ScaNN: Scalable Nearest Neighbors // Google Research : GitHub repository.
— URL:
<https://github.com/google-research/google-research/tree/master/scann> (дата обращения: 16.02.2026). — Текст: электронный.
16. Search filters // PhotoPrism Documentation. — URL:
<https://docs.photoprism.app/user-guide/search/filters/> (дата обращения: 19.02.2026). — Текст: электронный.

17. Search response // Microsoft Docs: GitHub repository. — URL: <https://github.com/MicrosoftDocs/bing-docs/blob/main/bing-docs/bing-image-search/how-to/search-response.md> (дата обращения: 17.02.2026). — Текст: электронный.
18. Searching // Immich Documentation. — URL: <https://docs.immich.app/features/searching/> (дата обращения: 18.02.2026). — Текст: электронный.
19. Similarity view // digiKam Documentation. — URL: https://docs.digikam.org/en/left_sidebar/similarity_view.html (дата обращения: 20.02.2026). — Текст: электронный.
20. Web Search API, REST: ImageSearch.SearchByImage // Yandex Cloud Documentation. — URL: <https://yandex.cloud/en/docs/search-api/api-ref/ImageSearch/searchByImage> (дата обращения: 13.02.2026). — Текст: электронный.
21. Крайнов А. 10 лет эволюции нейросетей в Яндекс Поиске: от поиска похожих картинок до мультимодальности в Нейро / А. Крайнов // Хабр. — 2024. — URL: <https://habr.com/ru/companies/yandex/news/863768/> (дата обращения: 12.02.2026). — Текст: электронный.
22. Aggarwal C. C. Recommender Systems: The Textbook. — Cham: Springer, 2016. — 498 p.
23. Manning C. D. Introduction to Information Retrieval / C. D. Manning, P. Raghavan, H. Schütze. — Cambridge: Cambridge University Press, 2008. — 482 p.
24. Cao B. Unifying deep local and global features for image search / B. Cao, A. Araujo, J. Sim // Computer Vision — ECCV 2020: proceedings. — Cham: Springer, 2020. — P. 726–743.
25. Dosovitskiy A. An image is worth 16x16 words: transformers for image recognition at scale / A. Dosovitskiy, L. Beyer, A. Kolesnikov [et al.] // International Conference on Learning Representations. — 2021.

26. Flickner M. Query by image and video content: the QBIC system / M. Flickner, H. Sawhney, W. Niblack [et al.] // Computer. — 1995. — Vol. 28, № 9. — P. 23–32.
27. He K. Deep residual learning for image recognition / K. He, X. Zhang, S. Ren, J. Sun // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. — 2016. — P. 770–778.
28. Hu H. Web-scale responsive visual search at Bing / H. Hu, Y. Wang, L. Yang [et al.] // Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. — 2018. — P. 359–367.
29. Kato T. Database architecture for content-based image retrieval // Image Storage and Retrieval Systems: proceedings SPIE. — 1992. — Vol. 1662. — P. 112–123.
30. Krizhevsky A. ImageNet classification with deep convolutional neural networks / A. Krizhevsky, I. Sutskever, G. E. Hinton // Advances in Neural Information Processing Systems. — 2012. — Vol. 25. — P. 1097–1105.
31. LeCun Y. Gradient-based learning applied to document recognition / Y. LeCun, L. Bottou, Y. Bengio, P. Haffner // Proceedings of the IEEE. — 1998. — Vol. 86, № 11. — P. 2278–2324.
32. Noh H. Large-scale image retrieval with attentive deep local features / H. Noh, A. Araujo, J. Sim, T. Weyand, B. Han // Proceedings of the IEEE International Conference on Computer Vision. — 2017. — P. 3456–3465.
33. Radford A. Learning transferable visual models from natural language supervision / A. Radford, J. W. Kim, C. Hallacy [et al.] // Proceedings of the 38th International Conference on Machine Learning. — 2021. — Vol. 139. — P. 8748–8763.
34. Simonyan K. Very deep convolutional networks for large-scale image recognition / K. Simonyan, A. Zisserman // International Conference on Learning Representations. — 2015.

35. Szegedy C. Going deeper with convolutions / C. Szegedy, W. Liu, Y. Jia [et al.] // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. — 2015. — P. 1–9.
36. Caron M. Emerging properties in self-supervised vision transformers / M. Caron, H. Touvron, I. Misra [et al.] // Proceedings of the IEEE/CVF International Conference on Computer Vision. — 2021. — P. 9650–9660.
37. Rekzi. Cat breeds // Hugging Face Datasets. — URL: <https://huggingface.co/datasets/Rekzi/cat-breeds> (дата обращения: 20.04.2026). — Текст: электронный.
38. Trofimov M. CBIR Image Similarity / M. Trofimov // GitHub. — URL: <https://github.com/MachineTrof/cbir-image-similarity> (дата обращения: 05.05.2026). — Текст: электронный.
39. Zheng H. Tiny ImageNet / H. Zheng // Hugging Face Datasets. — URL: <https://huggingface.co/datasets/zh-plus/tiny-imagenet> (дата обращения: 27.04.2026). — Текст: электронный.
40. Поиск по картинкам от Яндекса научился распознавать на изображениях товары и давать на них ссылки // Хабр. — 2021. — URL: <https://habr.com/ru/news/594689/> (дата обращения: 12.02.2026). — Текст: электронный.

ПРИЛОЖЕНИЕ 1. Примеры работы приложения

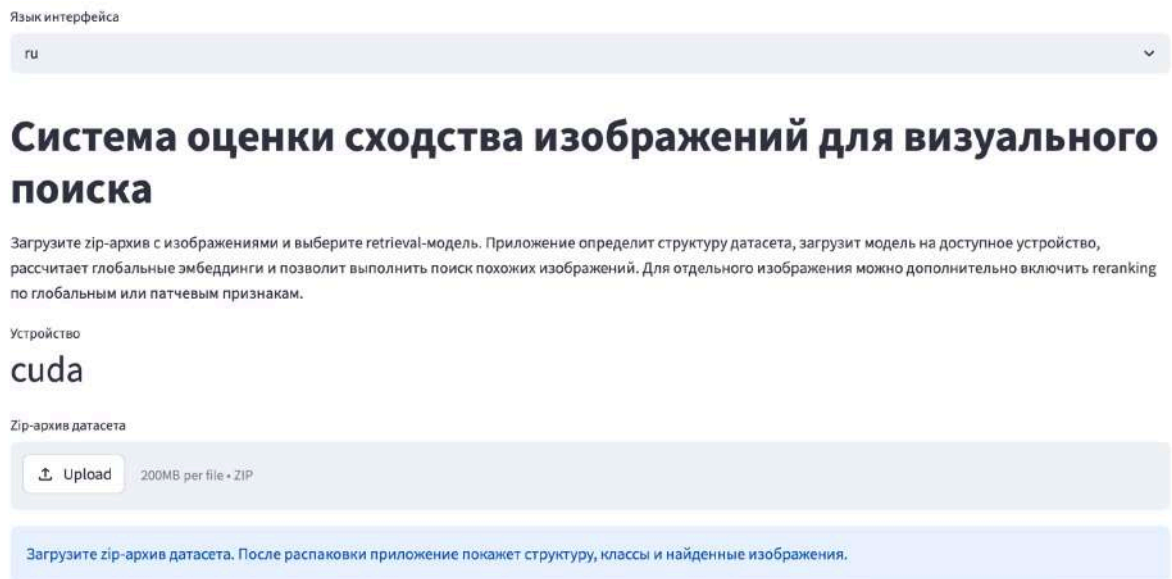


Рисунок 1. Начальный экран приложения

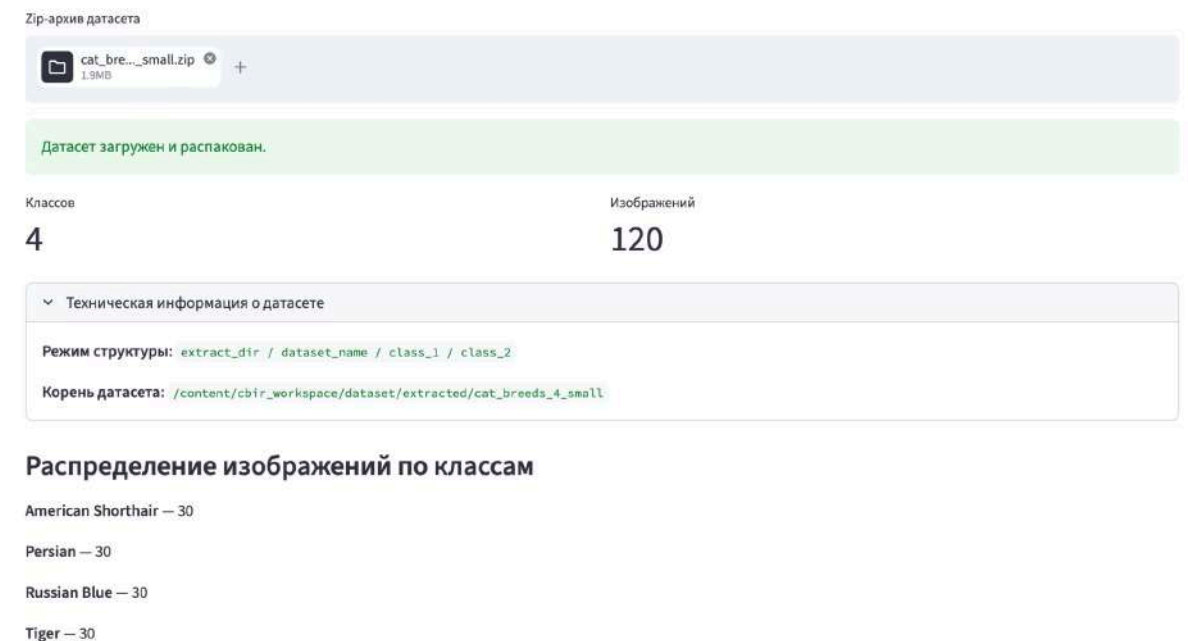


Рисунок 2. Информация о загруженном датасете

Просмотр изображений по классам

Класс:

American Shorthair

В классе American Shorthair найдено изображений: 30.

Количество изображений в галерее

10



Рисунок 3. Просмотр изображений выбранного класса

Retrieval-модель

Источник retrieval-модели

Загрузить .pth / .pt

Retrieval-модель (.pth / .pt)

Upload

200MB per file • PTH, PT

Для загружаемой .pth/.pt retrieval-модели ожидается full-extractor, сохранённый через torch.save(model, path). Метод forward должен возвращать тензор эмбедингов или структуру, из которой можно извлечь первый тензор.

Выберите или загрузите retrieval-модель для расчёта эмбедингов.

Рисунок 4. Панель выбора retrieval-модели

Retrieval-модель

Источник retrieval-модели

Загрузить .pth / .pt

Retrieval-модель (.pth / .pt)

 resnet1...ractor.pth 9.9 MB +

Для загружаемой .pth/.pt retrieval-модели ожидается full-extractor, сохранённый через `torch.save(model, path)`. Метод `forward` должен возвращать тензор эмбедингов или структуру, из которой можно извлечь первый тензор.

Параметры препроцессинга и инференса

Размер входного изображения для загружаемой retrieval-модели

128

Размер batch

32

Модель загружена.

Рассчитать эмбединги

Нажмите кнопку, чтобы рассчитать эмбединги по датасету.

Рисунок 5. Загрузка пользовательской модели, задание препроцессинга

Рассчитать эмбединги

Эмбединги датасета рассчитаны.

Размерность эмбединга

256

Время расчёта, сек.

1.2595

Изображений/сек.

95.2729

Секунд/изображение

0.010496

Рисунок 6. Результат расчёта эмбедингов

Изображение-запрос

Выберите изображение-запрос

American Shorthair/41383875_9939.jpg



Изображение-запрос

Поиск похожих изображений

Метод сравнения эмбедингов

Косинусное сходство

☒ L2-нормализация эмбедингов перед сравнением

Глубина выдачи top-k

10

Reranking для поиска по одному изображению

Источник reranking-модели

Без reranking

Рисунок 7. Выбор изображения-запроса, метода сравнения и глубины выдачи

Найти похожие изображения

Оценить весь датасет (retrieval)

Retrieval-метрики для выбранного изображения

Precision@10

0.3000

Recall@10

0.1034

Precision@1

0.0000

HitRate@10

1.0000

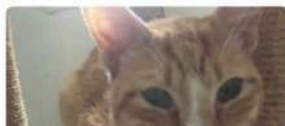
AP

0.2415

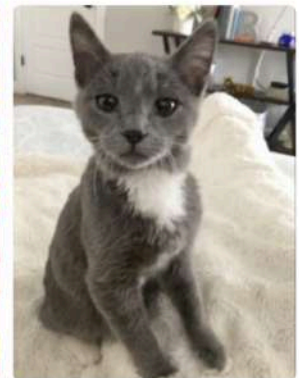


CosSim: 0.9043

Путь: Russian Blue/41287753_7376.jpg



CosSim: 0.8959



CosSim: 0.8936

Рисунок 8. Результат поиска по одному изображению без reranking-модели

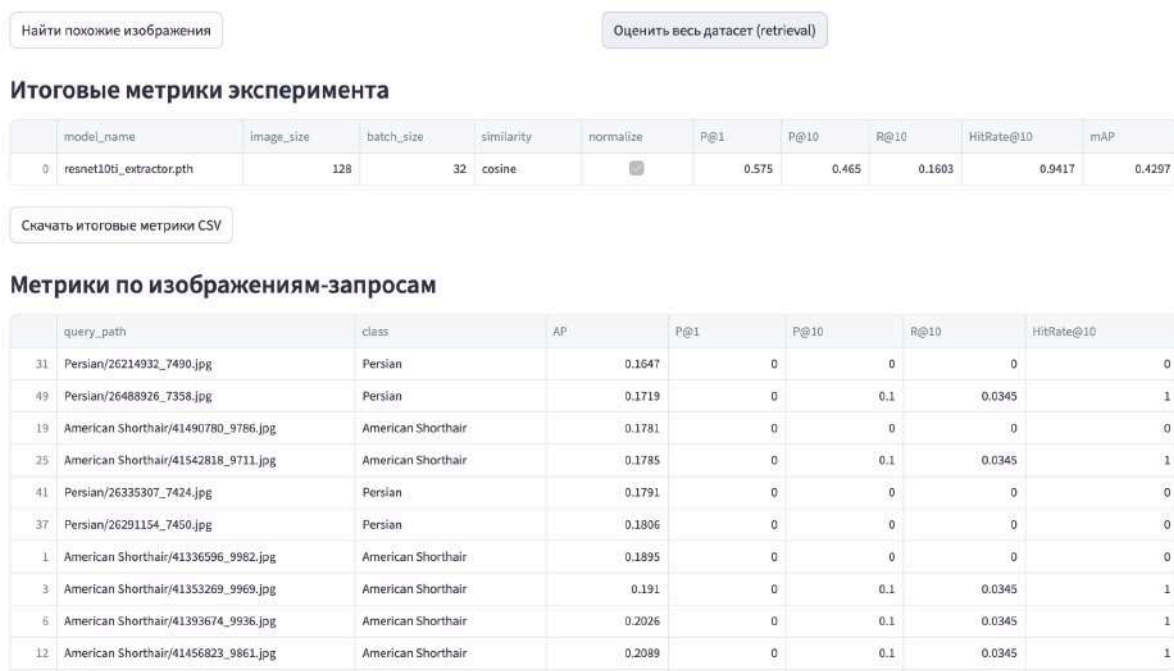


Рисунок 9. Таблицы итоговых метрик после полного обхода датасета

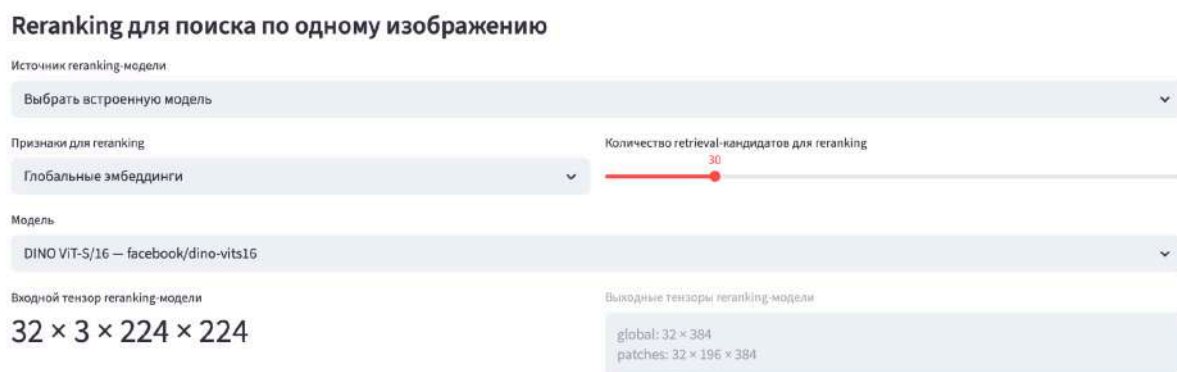


Рисунок 10. Настройка reranking-модели в режиме глобальных эмбедингов

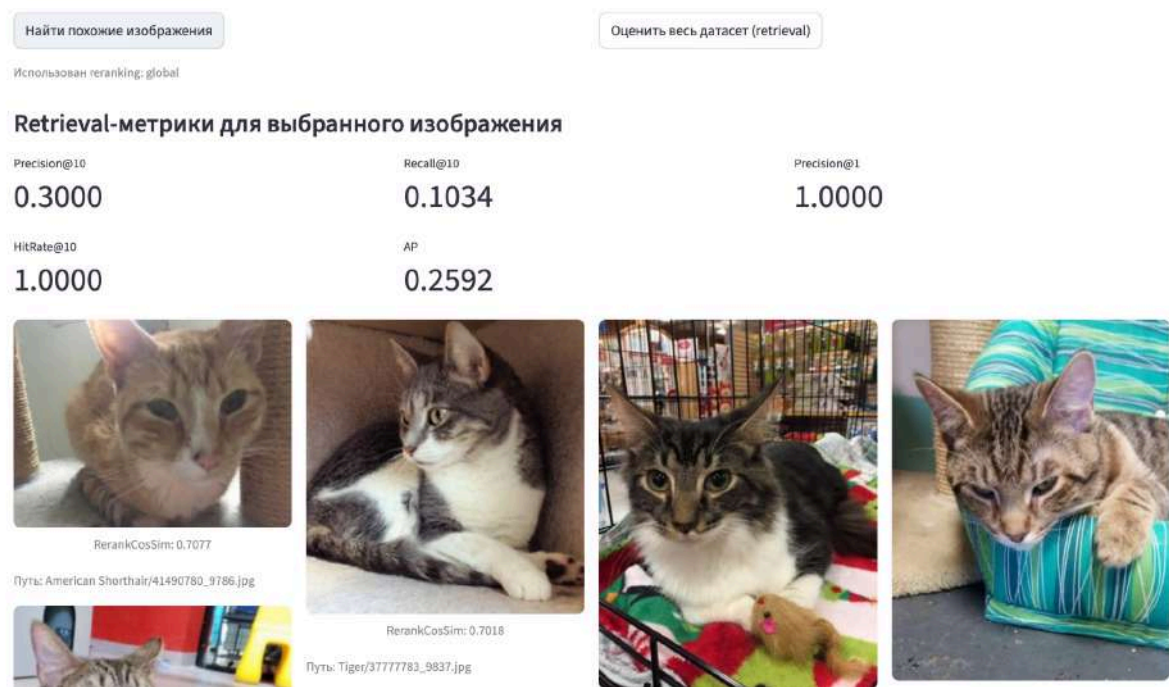


Рисунок 11. Результат поиска с reranking-шагом на глобальных эмбедингах

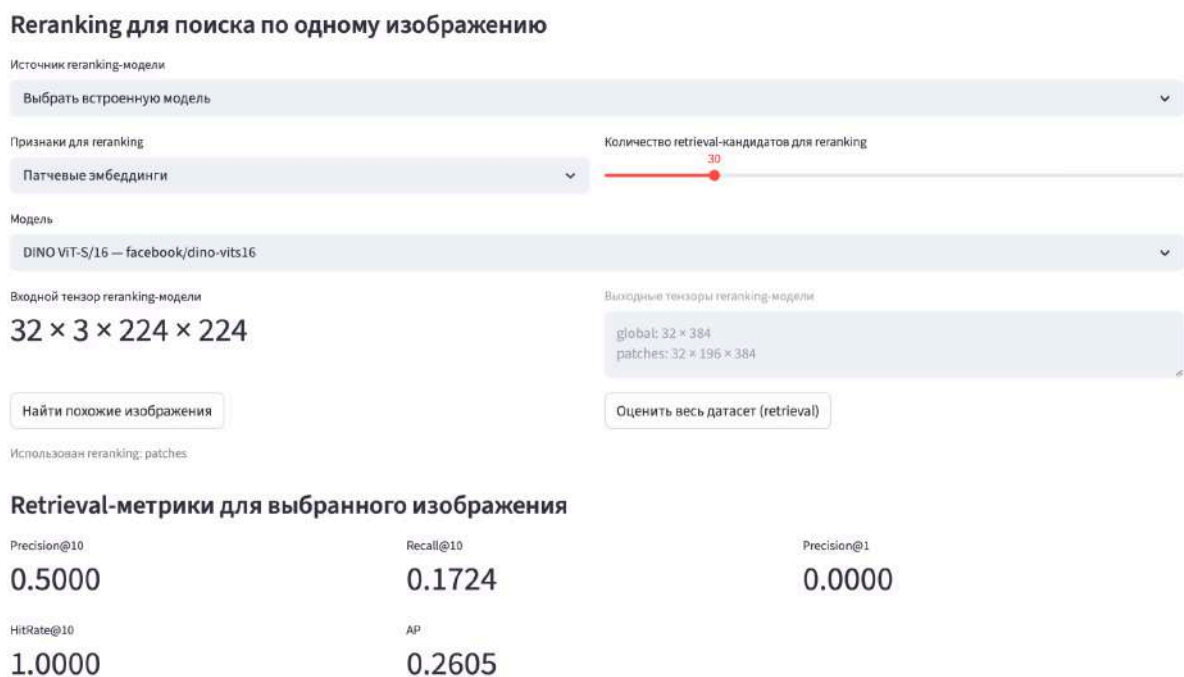


Рисунок 12. Настройка reranking-модели на патчевых эмбедингах

Retrieval-метрики для выбранного изображения

Precision@10
0.5000

Recall@10
0.1724

Precision@1
0.0000

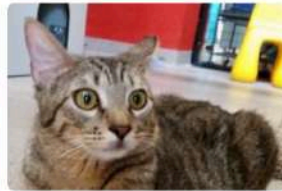
HitRate@10
1.0000

AP
0.2605



PatchMaxSim: 0.3993

Путь: Tiger/37718738_9902.jpg



PatchMaxSim: 0.3833

Путь: Tiger/37762967_9856.jpg



PatchMaxSim: 0.3761

Путь: Tiger/37702335_9918.jpg



PatchMaxSim: 0.3715

Путь: American Shorthair/41463425_9842.jpg

Рисунок 13. Результат поиска с reranking-шагом на патчевых эмбедингах

ПРИЛОЖЕНИЕ 2. Основной эксперимент

Таблица 1. Основной эксперимент на сравнение качества ранжирования
моделей

model_name	similarity	normalize	P@1	P@10	R@10	HitRate@10	mAP
DINOv2-B/14	euclidean	False	0,82625	0,8025	0,008	0,976	0,701464148
DINOv2-B/14	cosine	True	0,826	0,8031	0,008	0,976	0,699122123
DINOv2-B/14	dot	False	0,8205	0,801825	0,008	0,977	0,698374079
DINOv2-S/14	cosine	True	0,8175	0,78835	0,0079	0,97575	0,658950764
DINOv2-S/14	euclidean	False	0,81725	0,78705	0,0079	0,97675	0,658693024
DINOv2-S/14	dot	False	0,8125	0,788525	0,0079	0,975	0,658048174
DINO ViT-B/16	euclidean	False	0,79875	0,769	0,0077	0,9745	0,585972267
DINO ViT-B/16	cosine	True	0,79775	0,7682	0,0077	0,9755	0,583733001
DINO ViT-B/16	dot	False	0,7945	0,76685	0,0077	0,97575	0,579848748
DINO ViT-S/16	euclidean	False	0,79775	0,7694	0,0077	0,97425	0,576519749
DINO ViT-S/16	cosine	True	0,798	0,769025	0,0077	0,97375	0,575208556
DINO ViT-S/16	dot	False	0,80125	0,76855	0,0077	0,9765	0,572731294
ResNet50	cosine	True	0,776	0,73135	0,0073	0,976	0,572401575
ViT-B/16	dot	False	0,751	0,7085	0,0071	0,974	0,55951522
ViT-B/16	cosine	True	0,7745	0,7247	0,0073	0,97375	0,542033932
ViT-B/32	dot	False	0,7255	0,68665	0,0069	0,97625	0,538376928
ResNet50	euclidean	False	0,766	0,72395	0,0072	0,97575	0,527407427
ViT-B/32	cosine	True	0,74325	0,696025	0,007	0,97775	0,522233824
ViT-B/16	euclidean	False	0,76825	0,720275	0,0072	0,97625	0,510186643
ResNet18	cosine	True	0,75	0,710925	0,0071	0,974	0,504316685
ViT-B/32	euclidean	False	0,73775	0,694975	0,007	0,97875	0,493255215
ResNet50	dot	False	0,57625	0,5561	0,0056	0,956	0,49174785
ResNet18	euclidean	False	0,74025	0,698	0,007	0,97625	0,465765642
CLIP ViT-B/16	euclidean	False	0,747	0,6797	0,0068	0,98175	0,416863369
CLIP ViT-B/16	cosine	True	0,7445	0,6787	0,0068	0,984	0,41432366

CLIP ViT-B/32	euclidean	False	0,726	0,6705	0,0067	0,97975	0,413849133
CLIP ViT-B/32	cosine	True	0,73	0,672225	0,0067	0,98225	0,412083298
ResNet18	dot	False	0,56825	0,4396	0,0044	0,99725	0,385396748
CLIP ViT-B/16	dot	False	0,6235	0,56775	0,0057	0,98925	0,371070927
CLIP ViT-B/32	dot	False	0,65625	0,596275	0,006	0,985	0,370483226
ResNet10TI	cosine	True	0,589	0,52925	0,0053	0,96125	0,356684055
ResNet10TI	euclidean	False	0,57875	0,52365	0,0052	0,96425	0,352630391
ResNet10TI	dot	False	0,3575	0,367725	0,0037	0,9295	0,288526829